

# CS336 作业 2（系统）：系统和并行性

V版本 26.1。3

CS336工作人员

2026 年春季

## 1 作业概述

在此作业中，您将获得一些提高单 GPU 训练速度以及将训练扩展到多个 GPU 的实践经验。

您将实施什么

1. 基准测试和分析工具 2. 激活检查点 3  
. Flash Attention 2 Triton 内核 4. 分布式  
数据并行训练 5. 优化器状态分片 6. 完  
全分片数据并行训练

代码是什么样的

作业代码和本文可在 GitHub 上找到：

[github.com/stanford-cs336/assignment2-systems](https://github.com/stanford-cs336/assignment2-systems)

请使用 Git 克隆存储库。如果有任何更新，我们会通知您，您可以 `git pull` 获取最新信息。

1. `cs336-basics/`：在此作业中，您将分析我们在作业 1 中构建的一些组件。此文件夹包含作业 1 的人员解决方案代码，因此您将在此处找到 `cs336-`、`basics/pyproject.toml` 和 `cs336-basics/cs336_basics` 包。如果您想使用自己的模型实现，可以修改基目录中的 `pyproject.toml` 文件以指向您自己的包。

2. `/`： `cs336-systems` 基目录。我们创建了一个名为 `cs336_systems` 的空模块。请注意，这里没有代码，所以您应该能够从头开始做任何您想做的事情。

3. `tests/*.py`：该目录包含您必须通过的所有测试。这些测试调用 `tests/adapters.py` 中定义的钩子。您将实现适配器以将代码连接到测试。编写更多测试和/或修改测试代码有助于调试代码，但您的实现预计会通过原始提供的测试套件。

4. `README.md`：此文件包含有关预期目录结构的更多详细信息，以及有关设置环境的一些基本说明。

如何提交

您将向 Gradescope 提交以下文件：

- `writeup.pdf`：回答所有书面问题。请排版您的回复。
- `code.zip`：包含您编写的所有代码。

运行 `test_and_make_submission.sh` 中的脚本以创建 `code.zip` 文件。

## 2 分析和基准测试

在作业的第一部分中，我们将探索如何优化 Transformer 模型的性能，以最有效地利用 GPU。我们将分析我们的模型，以了解它在前向和后向传递过程中花费时间和内存的位置，然后使用自定义 GPU 内核优化自注意力操作，使其比常规 PyTorch 更快。在作业的后续部分中，我们将利用多个 GPU 并了解如何跨集群训练模型。

### 2.1 分析

在实施任何优化之前，首先分析我们的程序以了解它在哪里花费资源（例如时间和内存）会很有帮助。否则，我们可能会冒着优化模型中不占用大量时间或内存的部分的风险，因此看不到可衡量的端到端改进。

我们将实施三种绩效评估路径：

1. 使用 Python 标准库进行简单的端到端基准测试来计算前向和后向传递的时间
2. 使用 NVIDIA Nsight Systems 工具进行计算分析，以了解时间在 CPU 和 GPU 上的操作之间的分布情况
3. 内存分析

#### 2.1.1 设置 - 导入您的 **basics** 变压器模型

首先，我们要确保您可以从之前的任务中加载模型。在之前的作业中，我们在 Python 包中设置了模型，以便以后可以轻松导入。我们在 `./cs336-basics` 文件夹中添加了模型的参考实现，并在 `pyproject.toml` 文件中指向它。通过照常调用 `uv run [command]`，uv 将自动找到此本地 `cs336-basics` 包。如果您想使用自己的模型实现，可以修改 `pyproject.toml` 文件以指向您自己的包。

您可以使用以下方法测试是否可以导入模型：

```
$ uv run python
Using CPython 3.13.13
Creating virtual environment at: /path/to/uv/env/dir
  Built cs336-systems @ file:///path/to/systems/dir
  Built cs336-basics @ file:///path/to/basics/dir
Installed 78 packages in 168ms
Python 3.13.13 (main, Apr 7 2026, 20:49:46) [Clang 22.1.1 ] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> import cs336_basics
...
```

作业 1 中的相关模块现在应该可用（例如，对于 `model.py`，您可以使用 `import cs336_basics.model` 导入它）。

#### 2.1.2 模型大小

在整个任务中，我们将对模型进行基准测试和分析，以更好地了解其性能。为了了解事物如何大规模变化，我们将使用并参考以下模型配置。对于除排行榜之外的所有模型，我们将使用 10,000 的词汇量和 4 的批量大小，并具有不同的上下文长度。这项作业（以及以后的作业）将需要以表格和图表的形式呈现大量结果。我们强烈建议您自动构建表

你在代码中的写作，因为在 LaTeX 或 Typst 中格式化表格可能非常乏味。请参阅 `pandas.DataFrame.to_latex()` 和 `pandas.DataFrame.to_typst()` 或编写您自己的函数以根据您的首选的表格表示形式生成它们。

| Size   | d_model | d_ff  | num_layers | num_heads |
|--------|---------|-------|------------|-----------|
| small  | 768     | 3072  | 12         | 12        |
| medium | 1024    | 4096  | 24         | 16        |
| large  | 1280    | 5120  | 36         | 20        |
| xl     | 2560    | 10240 | 32         | 32        |
| 10B    | 4608    | 12288 | 50         | 36        |

表 1: 规格不同型号尺寸的on。这些大多是基于G PT-2 配置。

除非另有说明，否则使用上下文长度 512。

### 2.1.3 端到端基准测试

我们现在将实现一个简单的性能评估脚本。我们将测试模型的许多变体（更改精度、交换层等），因此让您的脚本通过命令行参数启用这些变体以使它们稍后易于运行是值得的。

首先，让我们通过计时前向传递、后向传递和优化器步骤来对模型进行最简单的分析。由于我们仅测量速度和内存，因此可以使用随机权重和数据。

衡量绩效是很微妙的——一些常见的陷阱可能会导致我们无法衡量我们想要的东西。对于 GPU 代码基准测试，需要注意的是 CUDA 调用为 *asynchronous*。当您调用 CUDA 内核时，例如调用 `torch.matmul` 时，PyTorch 函数调用会将控制权返回给您的代码，而无需等待矩阵乘法完成。这样，CPU 可以继续提前运行并调度新的运算，同时 GPU 完成矩阵乘法，这是性能上的重大胜利。另一方面，这意味着天真地测量 `torch.matmul` 调用需要多长时间才能返回并不能告诉我们 GPU 实际运行矩阵乘法需要多长时间。在 PyTorch 中，我们可以调用 `torch.cuda.synchronize()` 来等待所有调度的 GPU 内核完成，从而使我们能够更准确地测量 CUDA 内核运行时。该操作中的同步是指 CPU 运行时与 GPU 运行时同步。考虑到这一点，让我们编写基本的分析基础设施。

#### 问题 (benchmarking\_script): 基准测试脚本 (4 分)

(a) 编写一个脚本来对模型中的前向传递、后向传递和优化器步骤执行基本的端到端基准测试。具体来说，您的脚本应支持以下内容：

- 给定超参数（例如层数），初始化模型。
- 生成一批随机数据。
- 运行  $w$  预热步骤（在开始测量时间之前），然后对  $n$  步骤的执行进行计时（仅向前、向前和向后，或者使用优化器步骤向前和向后，具体取决于参数）。对于计时，您可以使用 Python `timeit` 模块（例如，使用 `timeit` 函数，或使用 `timeit.default_timer()`，它为您提供系统最高分辨率的时钟，因此是比 `time.time()` 更好的基准测试默认值）。

- 每一步后调用 `torch.cuda.synchronize()`。

可交付成果：一个脚本，它将使用给定的超参数初始化 `basics Transformer` 模型，创建一批随机数据，以及仅向前、向前和向后以及包含优化器步骤的完整训练步骤。

(b) 对第 2.1.2 节中描述的模型大小的前向、后向和优化器步骤进行计时。使用 5 个预热步骤并计算 10 个测量步骤的计时平均值和标准偏差。向前传球需要多长时间？向后传球怎么样？您是否发现测量结果的变异性很大，或者标准偏差很小？

可交付成果：用 1-2 句话回答您的时间安排。

(c) 基准测试的一个警告是不执行预热步骤。无需预热步骤即可重复您的分析。这对您的结果有何影响？您认为为什么会发生这种情况？还尝试通过 1 或 2 个预热步骤来运行脚本。为什么结果仍然会不同？

可交付成果：2-3 句话的回应。

#### 2.1.4 Nsight 系统分析器

端到端基准测试并不能告诉我们模型在前向和后向传递过程中花费了时间和内存，因此也没有暴露具体的优化机会。要知道我们的程序在每个组件（例如函数）上花费了多少时间，我们可以使用 *profiler*。执行分析器通过在函数开始和完成运行时插入防护来检测代码，从而可以提供函数级别的详细执行统计信息（例如调用次数、平均花费多长时间、在此函数上花费的累积时间等）。

标准 Python 分析器（例如 `cProfile`）无法分析 CUDA 内核，因为这些内核在 GPU 上异步执行。幸运的是，NVIDIA 提供了一个分析器，我们可以通过 CLI `nsys` 使用它。我们建议您从软件包管理器或使用下载页面的安装程序获取最新版本。在这部分作业中，您将使用 `nsys` 来分析 Transformer 模型的运行时。

使用 `nsys` 非常简单：运行上一节中的 Python 脚本，并在前面添加 `nsys profile`。例如，您可以使用以下命令运行脚本 `benchmark.py` 的基本配置文件：

```
$ uv run nsys profile -- python benchmark.py
```

然后，您可以使用 NVIDIA Nsight Systems 桌面应用程序在本地计算机上查看配置文件。在配置文件的 `CUDA API` 行中选择特定的 `CUDA API` 调用（在 CPU 上）将突出显示 `CUDA HW` 行中的所有相应内核执行（在 GPU 上）。

更全面的分析运行可能如下所示：

```
$ uv run nsys profile --trace=cuda,cudnn,cublas,osrt,nvtx --pytorch=functions-trace,autograd-shapes-nvtx --cudabacktrace=all --python-backtrace=cuda --gpu-metrics-devices=0 -- python benchmark.py
```

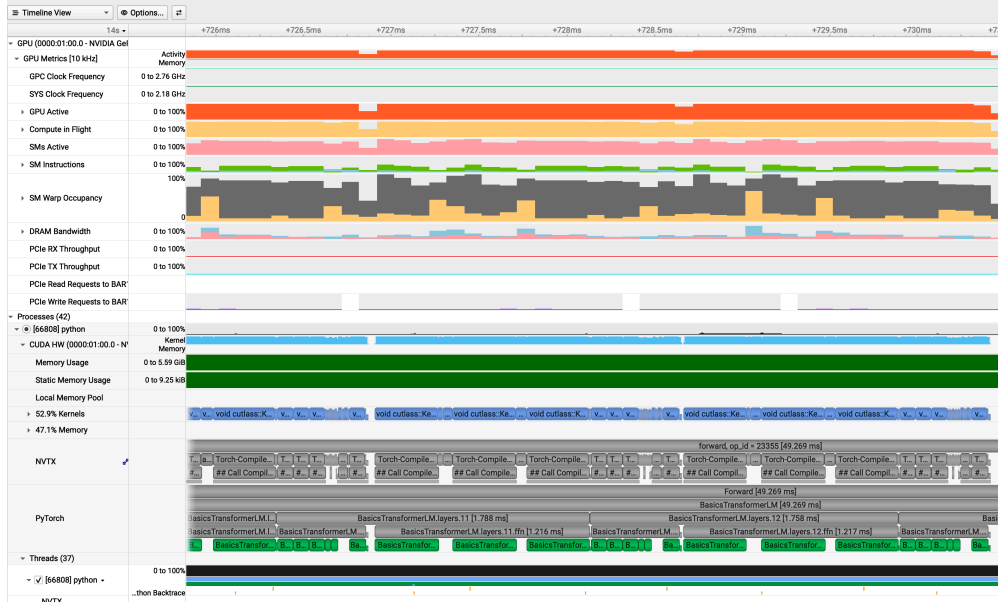


图 1: 详细的 Nsight Systems 跟踪

在此示例中, `--trace` 指定要记录哪些 API, `--pytorch` 在模块调用和自动分级期间插入 `nvtx` 标签, `--cudabacktrace` 和 `--python-backtrace` 提供更好的回溯以了解代码中调用给定内核的位置, 而 `--gpu-metrics-devices` 指定要测量哪个 GPU 的利用率。

向运行添加分析并不是免费的, 并且总体上会减慢您的运行速度。通常只值得在给定的运行中启用您正在寻找的功能。具体来说, 当不需要回溯时, 您可能需要删除 `--cudabacktrace=all` 和 `--python-backtrace=cuda`, 因为它们的开销太大。

我们鼓励您尝试使用 `nsys profile` 的各种命令行选项, 以了解它的功能。您还可以使用 NVTX 范围注释您的代码, 这些范围将在配置文件的 NVTX 行中显示为块, 捕获所有 CUDA API 调用和关联的内核执行。特别是, 您应该使用 NVTX 范围来忽略基准测试脚本中的预热步骤 (通过在配置文件中的 `nvtx` 标签上应用 `--nvtx-capture` 过滤器)。您还可以隔离哪些内核负责模型的前向和后向传递, 甚至可以通过如下注释您的实现来隔离哪些内核负责自注意力层的不同部分:

```
...
import torch.cuda.nvtx as nvtx

@nvtx.range("scaled dot product attention")
def annotated_scaled_dot_product_attention(
    ... # Q, K, V, mask
):
    ...
    with nvtx.range("computing attention scores"):
        ... # compute attention scores between Q and K

    with nvtx.range("computing softmax"):
        ... # compute softmax of attention scores

    with nvtx.range("final matmul"):
        ... # compute output projection
```

```
return ...
```

您可以通过以下方式将原始实现与基准测试脚本中带注释的版本交换：

```
cs336_basics.model.scaled_dot_product_attention = annotated_scaled_dot_product_attention
```

最后，值得注意的是，`torch.compile` 可能会让您很难将时间和资源归因于代码的特定部分。您可能需要在 `torch.compile` 和 `nvtx` 注释中包装和剥离代码的各个部分，才能将时间和资源使用情况正确归因于源代码的各个部分。

问题 `lem (nsys_profile)`: Nsight 系统分析 (5 点)

使用 `nsys` 分析您的前向传递、后向传递和优化器步骤，其中包含您选择的表 1 中的两个模型大小以及大于 128, 的三个二的幂上下文长度，其中最大可用大小应该是您可以容纳在内存中的最长上下文长度。选择您认为最有趣的组合。对于每个个人资料，请回答以下问题：

(a) 您的前传球总共花费了多少时间？它与我们之前使用 Python 标准库测量的结果相符吗？

可交付成果：1-2 句话的回应。

(b) 哪个 CUDA 内核在前向传播过程中占用最多的累积 GPU 时间？在模型的单次前向传递过程中，该内核被调用了多少次？当您进行向前和向后传递时，是否是占用最多运行时间的同一个内核？（提示：查看“统计系统视图”下的“CUDA GPU 内核摘要”，并使用 NVTX 范围进行过滤，以确定模型的哪些部分负责哪些内核。）

可交付成果：1-2 句话的回应。

(c) 尽管绝大多数 FLOP 发生在矩阵乘法中，但您会注意到其他几个内核仍然占用了相当大的整体运行时间。除了矩阵乘法之外，您还认为哪些其他内核在前向传播中占了重要的 CUDA 运行时间？

可交付成果：1-2 句话的回应。

(d) 分析通过 AdamW 的实现运行一个完整的训练步骤（即前向传递、计算损失并运行反向传递，最后是优化器步骤，就像您在训练期间所做的那样）。与进行推理（仅前向传递）相比，花在矩阵乘法上的时间比例有何变化？其他内核怎么样？

可交付成果：1-2 句话的回应。

(e) 比较前向传递过程中模型自注意力层内的 softmax 操作与矩阵乘法操作的运行时间。运行时间的差异与 FLOP 的差异相比如何？

可交付成果：1-2 句话的回应。

### 2.1.5 混合精度

到目前为止，我们一直在以 FP32 精度运行——所有模型参数和激活都具有 `torch.float32` 数据类型。然而，现代 NVIDIA GPU 包含专用 GPU 核心（Tensor Core），用于以较低精度加速矩阵乘法。例如，NVIDIA B200 规格表表示，其 FP32 的最大吞吐量为 80 TFLOPS，而 FP16（半精度浮点）或 BF16 (bfloat16) 的最大吞吐量则明显更高，高达 2500 TFLOPS。因此，使用较低精度的数据类型应该有助于我们加快训练和推理速度。

然而，天真地将我们的模型转换为较低精度的格式可能会降低模型的准确性。例如，实践中的许多梯度值通常太小而无法在 FP16 中表示，因此在使用 FP16 精度进行简单训练时会变为零。为了解决这个问题，在使用 FP16 进行训练时，通常会使用损失缩放——损失简单地乘以缩放因子，增加梯度幅度，这样它们就不会冲至零。此外，FP16 的动态范围比 FP32 低，这可能导致溢出，表现为 NaN 损失。完整的 bfloat16 训练通常更稳定（因为 BF16 具有与 FP32 相同的动态范围），但与 FP32 相比，仍然会影响最终模型的性能。

为了利用低精度数据类型的加速，通常使用 *mixed-precision* 训练。在 PyTorch 中，这是通过 `torch.autocast` 上下文管理器实现的。在这种情况下，某些运算（例如矩阵乘法）以较低精度的数据类型执行，而需要 FP32 完整动态范围的其他运算（例如累加和缩减）则保持原样。例如，以下代码将自动识别在前向传递期间以较低精度执行哪些操作，并将这些操作转换为指定的数据类型：

```
model : torch.nn.Module = ... # e.g. your Transformer model
dtype : torch.dtype = ... # e.g. torch.bfloat16
x : torch.Tensor = ... # input data

with torch.autocast(device_type="cuda", dtype=dtype):
    y = model(x)
```

如上所述，即使所累积的张量本身已被向下转换，通常保持更高精度的累积也是一个好主意。以下练习将帮助您建立关于为什么会出现这种情况的直觉。

问题 (**mixed\_precision\_accumulation**): 混合精度累加（1 分）

运行以下代码并评论结果的准确性。

```
s = torch.tensor(0, dtype=torch.float32)
for i in range(1000):
    s += torch.tensor(0.01, dtype=torch.float32)
print(s)

s = torch.tensor(0, dtype=torch.float16)
for i in range(1000):
    s += torch.tensor(0.01, dtype=torch.float16)
print(s)

s = torch.tensor(0, dtype=torch.float32)
for i in range(1000):
```

```

s += torch.tensor(0.01, dtype=torch.float16)
print(s)

s = torch.tensor(0, dtype=torch.float32)
for i in range(1000):
    x = torch.tensor(0.01, dtype=torch.float16)
    s += x.type(torch.float32)
print(s)

```

可交付成果：2-3 句话的回应。

现在，我们将首先将混合精度应用于玩具模型以构建直觉，然后应用于我们的基准测试脚本。

问题 (**benchmarking\_mixed\_precision**): 混合精度基准测试 (2 分)

(a) 考虑以下模型：

```

class ToyModel(nn.Module):
    def __init__(self, in_features: int, out_features: int):
        super().__init__()
        self.fc1 = nn.Linear(in_features, 10, bias=False)
        self.ln = nn.LayerNorm(10)
        self.fc2 = nn.Linear(10, out_features, bias=False)
        self.relu = nn.ReLU()

    def forward(self, x):
        x = self.relu(self.fc1(x))
        x = self.ln(x)
        x = self.fc2(x)
        return x

```

假设我们在 GPU 上训练模型，并且模型参数最初是 FP32 格式的。我们希望将自动铸造混合精度与 FP16 结合使用。有哪些数据类型：

- 自动投射上下文中的模型参数？
- 第一个前馈层 (`ToyModel.fc1`) 的输出？
- 层范数 (`ToyModel.ln`) 的输出？
- 模型的预测 logits？
- 损失？
- 模型的梯度？

可交付成果：上面列出的每个组件的数据类型。

(b) 您应该已经看到，FP16 混合精度自动转换对层归一化层的处理方式与前馈层不同。层归一化的哪些部分对混合精度敏感？如果我们使用 BF16 而不是 FP16，我们是否还需要以不同的方式对待层归一化？为什么或为什么不呢？

可交付成果：2-3 句话的回应。

(c) 修改基准测试脚本以选择使用 BF16 的混合精度运行模型。对于第 2.1.2 节中描述的每种语言模型大小，对使用和不使用混合精度的前向和后向传递进行计时。比较使用全精度与混合精度的结果，并评论模型大小变化时的任何趋势。您可能会发现 `nullcontext` 无操作上下文管理器很有用。

可交付成果：2-3 句话的回复，包括您的时间安排和评论。

## 2.1.6 分析内存

到目前为止，我们一直在关注计算性能。我们现在将注意力转移到 *memory*，这是语言模型训练和推理方面的另一个主要资源。PyTorch 还附带了一个强大的内存分析器，它可以跟踪一段时间内的分配情况。

要使用内存分析器，您可以修改基准测试脚本，如下所示：

```
... # warm-up phase in your benchmarking script

# Start recording memory history.
torch.cuda.memory._record_memory_history(max_entries=1000000)

... # what you want to profile in your benchmarking script

# Save a pickle file to be loaded by PyTorch's online tool.
torch.cuda.memory._dump_snapshot("memory_snapshot.pickle")

# Stop recording history.
torch.cuda.memory._record_memory_history(enabled=None)
```

这将输出一个文件 `memory_snapshot.pickle`，您可以将其加载到以下在线工具中：

[pytorch.org/memory\\_viz](https://pytorch.org/memory_viz)。该工具可让您查看总体内存使用时间线以及所做的每个单独分配，及其大小和指向其来源代码的堆栈跟踪。要使用此工具，您应该在 Web 浏览器中打开上面的链接，然后将 `pickle` 文件拖放到页面上。

您现在将使用 PyTorch 分析器来分析模型的内存使用情况。

### 问题 (`memory_profiling`): 内存分析 (4 分)

分析表 1 中上下文长度为 128 和 2048 的 `x1` 模型的前向传递、后向传递和优化器步骤的完整训练步骤。

(a) 在分析脚本中添加一个选项，以通过内存分析器运行模型。

重用以前的一些基础设施可能会有所帮助（例如，激活混合精度、加载特定模型大小等）。然后，运行脚本以在仅进行推理（仅前向传递）或完整训练步骤时获取 `x1` 模型的内存配置文件。你的记忆时间线是什么样的？您能根据您看到的峰值判断正在运行哪个阶段吗？

可交付成果：来自 `memory_viz` 工具的 `x1` 模型的“活动内存时间线”的两张图像：一张用于前向传递，一张用于运行完整的训练步骤（前向和反向传递，然后是优化器步骤），以及 2-3 句话响应。

(b) 进行前向传递时，每个上下文长度的峰值内存使用量是多少？进行完整训练步骤时怎么样？

可交付成果：每个上下文长度有两个数字的表格。

(c) 查找使用混合精度时  $x_l$  模型的前向传递和完整训练步骤的峰值内存使用量。混合精度是否会显著影响内存使用？

可交付成果：2-3 句话的回应。

(d) 考虑  $x_l$  模型。给定我们的参考超参数，Transformer 残差流中激活张量的大小是多少（单精度）？以 MiB 为单位给出此大小（即将字节数除以  $1024^2$ ）。

可交付成果：1-2 句话的回应以及你的推导。

(e) 现在仔细观察执行前向传递的  $x_l$  模型的内存快照的 [pytorch.org/memory\\_viz](https://pytorch.org/memory_viz) 中的“活动内存时间线”。当您降低“详细信息”级别时，该工具会隐藏相应级别的最小分配（例如，将“详细信息”设置为 10% 仅显示 10% 的最大分配）。显示的最大分配大小是多少？通过查看堆栈跟踪，您能看出这些分配来自哪里吗？

可交付成果：1-2 句话的回应。

(f) Nsight Systems 还具有用于内存分析的标志。您可以将它们与之前的 Nsight 标志结合起来，以了解模型生命周期中不同步骤发生的分配情况。使用 PyTorch 提供的 NVTX 标签来确定模型中的单个 TransformerBlock 为向后节省了多少内存（这些张量通常称为残差）。请注意 5 个贡献最大的操作，以及它们贡献的总内存百分比。

在向后传递期间，所有这些张量将被释放，但同时发出新的梯度张量。根据您的配置文件显示前向传递期间分配了多少内存，以及后向传递中每个 TransformerBlock 的内存使用变化量，计算为 TransformerBlock 生成的梯度张量占用多少内存。结果符合你的预期吗？

可交付成果：Nsight Systems 的屏幕截图和 1-2 段回复。

## 3 单 GPU 显存

本作业的后面部分将探讨在多个 GPU 上分割张量的技巧，但也有一些技巧甚至可以应用于单 GPU 训练。其中最常见的是梯度检查点（也称为激活检查点）。

### 3.1 Autograd 残差

回想一下，为了对模型执行向后传递，我们需要保存在前向传递中产生的激活。虽然对于某些操作来说显然是这种情况，但默认情况下，发生这种情况的情况会比您预期的要多得多。为向后传递保存的张量称为“残差”，或简称为“保存的张量”。

让我们对网络中保存的内容有一些了解。从我们不起眼的 RMSNorm 函数（为了简单起见，纯 FP32）开始，让我们为 autograd 保存或检索张量时添加一些钩子。

```

import torch
from torch import nn

x = torch.randn((4, 512, 2560), requires_grad=True)

class RMSNorm(nn.Module):
    def __init__(
        self,
        hidden_size: int,
        eps: float = 1e-5,
        device=None,
    ):
        super().__init__()
        self.weight = nn.Parameter(torch.ones(hidden_size, device=device))
        self.eps = eps

    def forward(self, x):
        rms = torch.rsqrt(x.pow(2).mean(-1, keepdim=True) + self.eps)
        x = x * rms
        return self.weight * x

def pack_hook(t):
    shape, dtype, grad_fn = t.shape, t.dtype, t.grad_fn
    print(f"Saving residual: {shape=}, {dtype=}, {grad_fn=}")
    return t

def unpack_hook(t):
    shape, dtype, grad_fn = t.shape, t.dtype, t.grad_fn
    print(f>Loading residual: {shape=}, {dtype=}, {grad_fn=}")
    return t

ln = RMSNorm(x.shape[-1])

with torch.autograd.graph.saved_tensors_hooks(pack_hook, unpack_hook):
    y = ln(x)
    y.sum().backward()

```

输出显示正在写出的张量数量令人担忧, 其中有几个处于完全激活大小!

```

$ uv run scripts/autograd_experiment.py
Saving residual: shape=torch.Size([4, 512, 2560]), dtype=torch.float32, grad_fn=None
Saving residual: shape=torch.Size([4, 512, 1]), dtype=torch.float32, grad_fn=<RsqrtBackward0
object at 0x7f7dd319b5e0>
Saving residual: shape=torch.Size([4, 512, 1]), dtype=torch.float32, grad_fn=<RsqrtBackward0
object at 0x7f7dd319b5e0>
Saving residual: shape=torch.Size([4, 512, 2560]), dtype=torch.float32, grad_fn=None
Saving residual: shape=torch.Size([4, 512, 2560]), dtype=torch.float32, grad_fn=<MulBackward0
object at 0x7f7dd319b5e0>
Saving residual: shape=torch.Size([2560]), dtype=torch.float32, grad_fn=None

Loading residual: shape=torch.Size([4, 512, 2560]), dtype=torch.float32,
grad_fn=<MulBackward0 object at 0x7f7cf14e6740>

```

```

Loading residual: shape=torch.Size([2560]), dtype=torch.float32, grad_fn=None
Loading residual: shape=torch.Size([4, 512, 1]), dtype=torch.float32, grad_fn=<RsqrtBackward0
object at 0x7f7cf14e6740>
Loading residual: shape=torch.Size([4, 512, 2560]), dtype=torch.float32, grad_fn=None
Loading residual: shape=torch.Size([4, 512, 1]), dtype=torch.float32, grad_fn=<RsqrtBackward0
object at 0x7f7cf14e6740>
Loading residual: shape=torch.Size([4, 512, 2560]), dtype=torch.float32, grad_fn=None

```

### 3.1.1 算子融合

在这种情况下，很明显所使用的操作的粒度太高。我们想要一个单一的操作，它接受 RMSNorm 权重和激活，并吐出输出，并且该操作在向后传递中是单一的。这是内核融合的动机之一。由于 RMSNorm 的表现相当良好，我们甚至可以使用 `torch.compile` 自动融合它。

```

...
ln = torch.compile(RMSNorm(x.shape[-1]))

with torch.autograd.graph.saved_tensors_hooks(pack_hook, unpack_hook):
    y = ln(x)
    y.sum().backward()

```

新的输出明显更好：

```

Saving residual: shape=torch.Size([4, 512, 2560]), dtype=torch.float32, grad_fn=None
Saving residual: shape=torch.Size([2560]), dtype=torch.float32, grad_fn=None
Saving residual: shape=torch.Size([4, 512, 1]), dtype=torch.float32, grad_fn=None
Loading residual: shape=torch.Size([4, 512, 2560]), dtype=torch.float32, grad_fn=None
Loading residual: shape=torch.Size([2560]), dtype=torch.float32, grad_fn=None
Loading residual: shape=torch.Size([4, 512, 1]), dtype=torch.float32, grad_fn=None

```

我们只需要为后向传递保存一个全尺寸激活张量，即 RMSNorm 函数的输入。另请注意，加载顺序不再与保存顺序相反，并且每个残差不再具有 `grad_fn` 依赖项 - PyTorch 将整个 RMSNorm 视为单个函数。

## 3.2 激活检查点

虽然融合无疑很有用，但它在节省内存方面只能让我们到目前为止。例如，让我们融合一个大小为 `x1` 的 TransformerBlock。

```

import torch
from cs336_basics.model import RotaryEmbedding, TransformerBlock

# num_layers for this model is 32
d_model, d_ff, num_heads, context_length = 2560, 10240, 16, 2048
block = TransformerBlock(d_model=d_model, d_ff=d_ff, num_heads=num_heads,
    positional_encoder=RotaryEmbedding(dim=d_model // num_heads, context_length=context_length))

# Fuse as much torch.compile will allow
block = torch.compile(block, fullgraph=True)
x = torch.randn((4, context_length, d_model), requires_grad=True)

```

```

...
# Now logs the number of bytes saved
total_size_bytes = 0
def pack_hook(t):
    if isinstance(t, torch.nn.Parameter): # Skip logging parameters to avoid double counting
        return t
    global total_size_bytes
    shape, dtype, grad_fn = t.shape, t.dtype, t.grad_fn
    total_size_bytes += t.numel() * t.element_size()
    print(f"Saving residual: {shape=}, {dtype=}, {grad_fn=}")
    return t

...

# Run forward pass, saving for backward
with torch.autograd.graph.saved_tensors_hooks(pack_hook, unpack_hook):
    y = block(x)

print(f"Total size of saved tensors in single TransformerBlock: {total_size_bytes /
(1024**2):.2f} MiB")

```

该脚本向我们显示了我们为向后节省了多少内存：

```

...
Total size of saved tensors in single TransformerBlock: 3651.31 MiB

```

每层 3.6 GiB。如果我们对所有层都执行此操作，我们将获得 114 GiB 的激活，只是保存用于向后！注意力操作的残差存在大量浪费，我们将在第 4 节中修复该问题，但即使进行此修复，内存使用量也会随着批量大小、序列长度和嵌入大小线性增长。

### 3.2.1 重新计算

可以只保存结果的定期检查点，并重新计算中间的值，而不是保留我们生成的每个张量。PyTorch 有一个接口，我们可以调用它以简单的方式处理这个问题。torch.utils.checkpoint.checkpoint 接受一个函数以及该函数的参数。然后它修改传递的函数的行为：

#### 1. 在前向传播中：

1. 将输入值保存到函数中
2. 禁止在前向传递中保存张量

#### 2. 在向后传递中：

1. 预先准备一个重新计算步骤，其中根据之前保存的输入重新计算前向传递，并保存后向传递的值
2. 运行反向传递并且可以释放所有张量

在运行 4 个 Transformer 块的简单情况下，我们看到内存加起来正如我们预期的那样。

```

...
def four_blocks(x):
    x = block(x)

```

```

    x = block(x)
    x = block(x)
    x = block(x)
    return x

with torch.autograd.graph.saved_tensors_hooks(pack_hook, unpack_hook):
    y = four_blocks(x)

print(f"Total size of saved tensors in four TransformerBlocks: {total_size_bytes /
(1024**2):.2f} MiB")

```

```
Total size of saved tensors in four TransformerBlocks: 14605.25 MiB
```

但我们可以采用梯度检查点，如下所示：

```

from torch.utils.checkpoint import checkpoint
def two_blocks(x):
    x = block(x)
    x = block(x)
    return x

def four_blocks_checkpoint(x):
    # checkpoint throws out all the saved tensors until the backward pass
    # when getting to the checkpointed block in the backward pass,
    # it reruns a forward pass to produce the saved tensors,
    # then completes normal backward pass.
    x = checkpoint(two_blocks, x, use_reentrant=False)
    x = checkpoint(two_blocks, x, use_reentrant=False)
    return x

with torch.autograd.graph.saved_tensors_hooks(pack_hook, unpack_hook):
    y = four_blocks_checkpoint(x)

print(f"Total size of saved tensors in four TransformerBlocks with checkpointing:
{total_size_bytes / (1024**2):.2f} MiB")

```

```

Saving residual: shape=torch.Size([0]), dtype=torch.float32, grad_fn=None
Saving residual: shape=torch.Size([4, 2048, 2560]), dtype=torch.float32, grad_fn=None
Saving residual: shape=torch.Size([0]), dtype=torch.float32, grad_fn=None
Saving residual: shape=torch.Size([4, 2048, 2560]), dtype=torch.float32,
grad_fn=<torch.autograd.function.CompiledFunctionBackward object at 0x7aa0657a19d0>
Total size of saved tensors in four TransformerBlocks with checkpointing: 160.00 MiB

```

请记住，这并没有消除内存使用。相反，它将我们的内存使用分为两类：我们保存的长期存储，以准备在每个 checkpoint 调用的入口点（检查点本身）进行重新计算，以及在检查点块内的重新计算过程中生成的短期内存，以方便向后传递。由于我们主要考虑的是峰值内存使用量，因此我们希望平衡已保存检查点的内存成本与实现完整块残差的内存成本。我们使用的检查点越多，单个块内的物化内存量就越小，但检查点本身需要的内存就越多。

在上面的示例中，很明显，重新计算的残差内存占主导地位（部分原因是我们的检查点很小），因此将平衡更多地转向检查点内存将是有益的。因此，我们希望减小检查点块的范围。

拥有更大或更小的检查点块不会影响重新计算的计算成本，但我们可以通过使用 *recursive checkpointing* 以计算成本继续减少所需的内存，这意味着我们将 checkpoint 调用嵌套在其他 checkpoint 调用中。

#### 问题（`gradient_checkpointing`）： 内存优化梯度检查点（4分）

考虑一个 Transformer，它具有按顺序堆叠的  $N$  个相同块。在没有任何检查点的情况下，所有  $N$  块的残差值同时保持活动状态，从而提供  $O(N)$  峰值激活内存。我们可以腾出手来将前向传递的任何子集包装在 checkpoint 中，包括相互嵌套 checkpoint 调用。

(a) 什么检查点策略可以最小化峰值激活内存，忽略计算成本？描述如何安排 checkpoint 调用（代码草图即可），并给出渐近峰值激活内存和策略计算作为  $N$  的函数。假设单个块保存的残差主导任何每个检查点的簿记。

可交付成果：对该策略及其渐近峰值记忆的 3-5 句话描述，以及简短的代码草图。

(b) 考虑批量大小为 4 且序列长度为 2048 的 xl 模型配置，如上所述。如果您只有时间/计算预算来运行一步重新计算（这意味着您可能无法嵌套 checkpoint 调用），那么减少峰值内存的最佳检查点策略是什么？分析您跑步的峰值内存以验证您的假设。比较下一个更小和更大的检查点块大小的峰值内存以确定。

可交付成果：用 3-5 句话描述您的推理以及针对您的策略测得的峰值记忆。

## 4 个 GPU 内核

### 4.1 使用 FlashAttention-2 优化注意力

#### 4.1.1 PyTorch 注意力基准测试

您的分析可能表明您的注意力层在内存和计算方面都有优化的机会。在较高层次上，注意力运算由矩阵乘法、softmax 和另一个矩阵乘法组成：

$$\text{Attention}(Q, K, V) = \text{softmax} \left( \text{mask} \left( \frac{QK^T}{\sqrt{d_k}} \right) \right) V \quad (1)$$

朴素的注意力实现需要为每个批/头元素保存形状为  $\text{seq\_len} \times \text{seq\_len}$  的注意力分数矩阵，该矩阵可能会随着长序列长度而变得非常大，从而导致任何具有长输入或输出的任务出现内存不足错误。我们将按照 Flash Attention-2 论文实现一个注意力内核，该内核按图块计算注意力，并避免显式具体化  $\text{seq\_len} \times \text{seq\_len}$  注意力分数矩阵，从而能够扩展到更长的序列长度。

#### 问题 (pytorch\_attention): PyTorch 注意力基准测试 (2 分)

(a) 对不同规模的注意力实施情况进行基准测试。编写一个脚本，该脚本将：

(i) 将批量大小固定为 8，并且不使用多头注意力（即删除头维度）。(ii) 迭代头嵌入维度  $d_{\text{model}}$  的 [16, 32, 64, 128] 和序列长度的 [256, 1024, 4096, 8192, 16384] 的笛卡尔积。(iii) 创建适当大小的随机输入  $Q, K, V$ 。(iv) 时间 100 使用输入向前传递注意力。(v) 测量向后传递开始之前以及向后传递 100 次之前使用了多少内存。(vi) 确保热身，并在每次向前/向后传递后调用 `torch.cuda.synchronize()`。

根据您的 GPU，其中一些配置预计会耗尽内存。报告这些配置的计时（或内存不足错误）。内存不足错误的大小是多少？在您发现内存耗尽的最小配置之一中计算注意力的内存使用情况（您可以使用作业 1 中 Transformer 的内存使用情况方程）。向后节省的内存如何随序列长度变化？您会采取什么措施来消除这种内存成本？

可交付成果：一张表格，其中包含您的计时、内存使用情况的计算以及 1-2 段的响应。

## 4.2 对 JIT 编译的注意力进行基准测试

从 2.0 版开始，PyTorch 还附带了一个强大的即时编译器，它会自动尝试对 PyTorch 函数应用许多优化：请参阅

[https://pytorch.org/tutorials/intermediate/torch\\_compile\\_tutorial.html](https://pytorch.org/tutorials/intermediate/torch_compile_tutorial.html) 进行介绍。特别是，它将尝试通过动态分析您的计算图来自动生成融合的 Triton 内核。使用 PyTorch 编译器的界面非常简单。例如，如果我们想将其应用到模型的单层，我们可以使用：

```
layer = SomePyTorchModule(...)
compiled_layer = torch.compile(layer)
```

现在，`compiled_layer` 的功能行为与 `layer` 类似（例如，具有向前和向后传递）。我们还可以使用 `torch.compile(model)` 编译整个 PyTorch 模型，甚至是调用 PyTorch 操作的 Python 函数。

#### 问题 (torch\_compile): Torch 编译 (2 分)

(a) 扩展注意力基准测试脚本以包含 PyTorch 注意力实现的编译版本，并将其性能与具有与上述 `pytorch_attention` 问题相同配置的未编译版本进行比较。

可交付成果：一张表格，将编译后的注意力模块的前向和后向传递时间与上述 `pytorch_attention` 问题的未编译版本进行比较。

(b) 现在，在端到端基准测试脚本中编译整个 Transformer 模型。前向传球的性能如何变化？组合的前向和后向传递以及优化器步骤怎么样？

可交付成果：比较原始 Transformer 模型和编译后的 Transformer 模型的表格。

考虑到我们所看到的关于序列长度的缩放行为，我们需要进行重大改进来处理大型序列。即使使用 `torch.compile`，当前的实现在长序列长度下也会遇到非常糟糕的内存访问模式。为此，我们将编写 FlashAttention-2 的 Triton 实现，在这里我们可以更好地控制内存的访问方式以及何时计算什么。

#### 4.2.1 示例 - 加权和

为了介绍您需要了解的有关 Triton 的知识以及它如何与 PyTorch 互操作，我们将通过一个示例内核来进行“加权和”运算。有关快速使用 Triton 的更多资源，请参阅 Triton 的教程。我们注意到，这些教程没有使用新的、方便的块指针抽象，我们将在下面介绍。

给定一个输入矩阵  $X$ ，我们将其条目乘以列权重向量  $w$ ，并对每一行求和，得到  $X$  和  $w$  的矩阵向量乘积。我们将首先完成此操作的前向传递，然后编写用于后向传递的 Triton 内核。

前传

前进帕 我们内核的 `ss` 就是下面广播的内部 `p` 产品。

```
def weighted_sum(x, weight):  
    # Here, assume that x has n-dim shape [..., D], and weight has 1D shape [D]  
    return (weight * x).sum(axis=-1)
```

在编写 Triton 内核时，我们将让每个程序实例（可能并行运行）计算 *tile* 行  $x$  的加权和，并将相应的标量输出写入输出张量。在 Triton 中，程序实例是运行同一程序的线程块，这些 *thread blocks* 可以在 GPU 上并行运行。我们不将 *tensors* 作为参数，而是将 *pointers* 作为其第一个元素，并对每个张量使用 *strides* 来告诉我们如何沿轴移动。

我们可以使用步幅来加载与我们在运行实例中求和的  $x$  行图块相对应的张量，并使用程序 ID 来划分工作（即，实例  $i$  将处理  $x$  行的第  $i$  个图块）。在这个简单的情况下，Triton 和 PyTorch 中的前向传递之间的主要区别是需要进行指针算术和显式加载/存储。我们将使用 `tl.make_block_ptr` 的块指针抽象来极大地简化指针运算，尽管这意味着我们需要做一些设置来准备块指针。

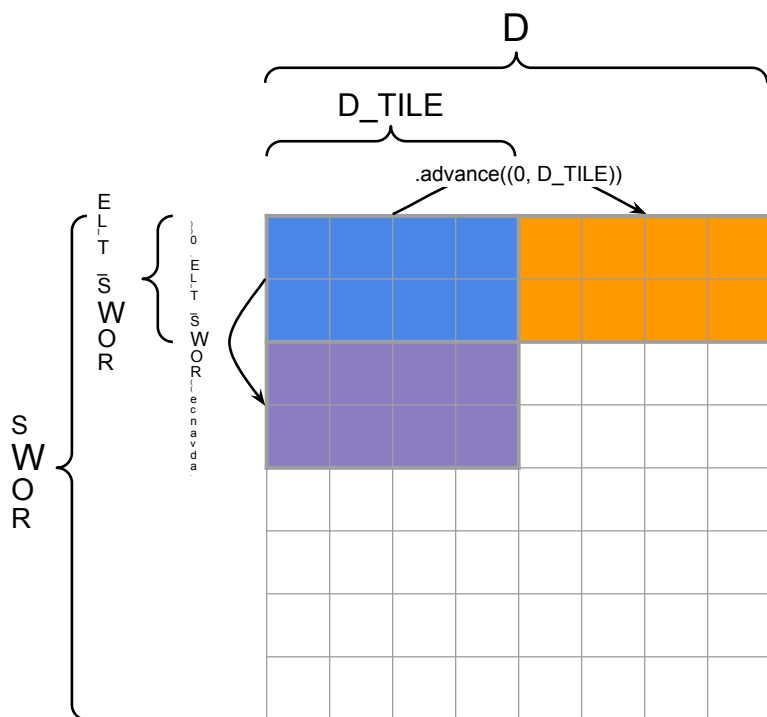


图 2: 平铺和在加权和内核 ex 中推进块指针 充足 (第 4.2.1 节)。

请参阅图 2, 了解平铺示意图以及块指针如何前进。上面的加权和函数如下所示:

```
import triton
import triton.language as tl

@triton.jit
def weighted_sum_fwd(
    x_ptr, weight_ptr, # Input pointers
    output_ptr, # Output pointer
    x_stride_row, x_stride_dim, # Strides tell us how to move one element in each axis of a tensor
    weight_stride_dim, # Likely 1
    output_stride_row, # Likely 1
    NUM_ROWS, D,
    ROWS_TILE_SIZE: tl.constexpr, D_TILE_SIZE: tl.constexpr, # Tile shapes must be known at compile time
):
    # Each instance will compute the weighted sum of a tile of rows of x.
    # `tl.program_id` gives us a way to check which thread block we're running in
    row_tile_idx = tl.program_id(0)

    # Block pointers give us a way to select from an ND region of memory
    # and move our selection around.
    # The block pointer must know:
    # - The pointer to the first element of the tensor
    # - The overall shape of the tensor to handle out-of-bounds access
    # - The strides of each dimension to use the memory layout properly
    # - The ND coordinates of the starting block, i.e., "offsets"
    # - The block shape to load/store at a time
    # - The order of the dimensions in memory from major to minor
    # axes (= np.argsort(strides)) for optimizations, needed for
    # TMA support on >=Hopper
```

```

x_block_ptr = tl.make_block_ptr(
    x_ptr,
    shape=(NUM_ROWS, D,),
    strides=(x_stride_row, x_stride_dim),
    offsets=(row_tile_idx * ROWS_TILE_SIZE, 0),
    block_shape=(ROWS_TILE_SIZE, D_TILE_SIZE),
    order=(1, 0),
)

weight_block_ptr = tl.make_block_ptr(
    weight_ptr,
    shape=(D,),
    strides=(weight_stride_dim,),
    offsets=(0,),
    block_shape=(D_TILE_SIZE,),
    order=(0,),
)

output_block_ptr = tl.make_block_ptr(
    output_ptr,
    shape=(NUM_ROWS,),
    strides=(output_stride_row,),
    offsets=(row_tile_idx * ROWS_TILE_SIZE,),
    block_shape=(ROWS_TILE_SIZE,),
    order=(0,),
)

# Initialize a buffer to write to
output = tl.zeros((ROWS_TILE_SIZE,), dtype=tl.float32)

for i in range(tl.cdiv(D, D_TILE_SIZE)):
    # Load the current block pointer
    # Since ROWS_TILE_SIZE might not divide NUM_ROWS, and D_TILE_SIZE might not divide D,
    # we need boundary checks for both dimensions
    row = tl.load(x_block_ptr, boundary_check=(0, 1), padding_option="zero") # (ROWS_TILE_SIZE, D_TILE_SIZE)
    weight = tl.load(weight_block_ptr, boundary_check=(0,), padding_option="zero") # (D_TILE_SIZE,)

    # Compute the weighted sum of the row.
    output += tl.sum(row * weight[None, :], axis=1)

    # Move the pointers to the next tile.
    # These are (rows, columns) coordinate deltas
    x_block_ptr = x_block_ptr.advance((0, D_TILE_SIZE)) # Move by D_TILE_SIZE in the last dimension
    weight_block_ptr = weight_block_ptr.advance((D_TILE_SIZE,)) # Move by D_TILE_SIZE

# Write output to the output block pointer (a single scalar per row).
# Since ROWS_TILE_SIZE might not divide NUM_ROWS, we need boundary checks
tl.store(output_block_ptr, output, boundary_check=(0,))

```

现在让我们将此内核包装在 PyTorch Autograd 函数中，该函数将与 PyTorch 进行互操作（即，将 Tensor 作为输入，输出 Tensor，然后在向后传递过程中也使用 autograd 引擎）：

```

class WeightedSumFunc(torch.autograd.Function):
    @staticmethod
    def forward(ctx, x, weight):
        # Cache x and weight to be used in the backward pass, when we
        # only receive the gradient wrt. the output tensor, and
        # need to compute the gradients wrt. x and weight.
        D, output_dims = x.shape[-1], x.shape[:-1]

        # Reshape input tensor to 2D
        input_shape = x.shape
        x = rearrange(x, "... d -> (...) d")

        ctx.save_for_backward(x, weight)

```

```

assert len(weight.shape) == 1 and weight.shape[0] == D, "Dimension mismatch"
assert x.is_cuda and weight.is_cuda, "Expected CUDA tensors"
assert x.is_contiguous(), "Our pointer arithmetic will assume contiguous x"

ctx.D_TILE_SIZE = triton.next_power_of_2(D) // 16 # Roughly 16 loops through the embedding dimension
ctx.ROWS_TILE_SIZE = 16 # Each thread processes 16 batch elements at a time
ctx.input_shape = input_shape

# Need to initialize empty result tensor. Note that these elements are not necessarily 0!
y = torch.empty(output_dims, device=x.device)

# Launch our kernel with n instances in our 1D grid.
n_rows = y.numel()
weighted_sum_fwd[(triton.cdiv(n_rows, ctx.ROWS_TILE_SIZE),)](
    x, weight,
    y,
    x.stride(0), x.stride(1),
    weight.stride(0),
    y.stride(0),
    NUM_ROWS=n_rows, D=D,
    ROWS_TILE_SIZE=ctx.ROWS_TILE_SIZE, D_TILE_SIZE=ctx.D_TILE_SIZE,
)

return y.view(input_shape[:-1])

```

请注意，当我们使用 `weighted_sum_fwd[(triton.cdiv(n_rows, ctx.ROWS_TILE_SIZE),)]` 调用 Triton 内核时，我们通过传递元组 `(triton.cdiv(n_rows, ctx.ROWS_TILE_SIZE),)` 来定义所谓的线程块“启动网格”。然后，我们可以在内核中使用 `tl.program_id(0)` 访问线程块索引。

## 向后传球

既然我们定义了我们的内核，我们还需要编写自己的 `backward` 函数。

在前向传递中，我们获得了该层的输入，并且需要计算其输出。在向后传递中，回想一下，我们将获得目标相对于输出的梯度，并且需要计算相对于每个输入的梯度。在我们的例子中，我们的操作有一个矩阵  $x: \mathbb{R}^{n \times h}$  和一个权重向量  $w: \mathbb{R}^h$  作为输入。为了简洁起见，我们将操作称为  $f(x, w)$ ，其范围为  $\mathbb{R}^n$ 。然后，假设我们给定  $\nabla_{f(x, w)} \mathcal{L}$ ，即损失  $\mathcal{L}$  相对于层输出的梯度，我们可以应用多元链式法则来获得以下关于  $x$  和  $w$  的梯度表达式：

$$(\nabla_x \mathcal{L})_{ij} = \sum_{k=1}^n \frac{\partial f(x, w)_k}{\partial x_{ij}} (\nabla_{f(x, w)} \mathcal{L})_k = w_j \cdot (\nabla_{f(x, w)} \mathcal{L})_i \quad (2)$$

$$(\nabla_w \mathcal{L})_j = \sum_{i=1}^n \frac{\partial f(x, w)_i}{\partial w_j} (\nabla_{f(x, w)} \mathcal{L})_i = \sum_{i=1}^n x_{ij} \cdot (\nabla_{f(x, w)} \mathcal{L})_i \quad (3)$$

这给出了计算反向传播的简单公式。为了获得相对于  $x$  的后退步，我们应用方程 2 并取  $w$  和  $\nabla_{f(x, w)}$  的外积。要计算相对于  $w$  (即  $(\nabla_w \mathcal{L})_j$ ) 的后向步长，我们必须将输入梯度乘以相应的输出行。

我们的向后传递内核将首先定义所有块指针，然后计算  $\nabla_x \mathcal{L}$ ：

```

@triton.jit
def weighted_sum_backward(
    x_ptr, weight_ptr, # Input
    grad_output_ptr, # Grad input
    grad_x_ptr, partial_grad_weight_ptr, # Grad outputs
    stride_xr, stride_xd,

```

```

stride_wd,
stride_gr,
stride_gxr, stride_gxd,
stride_gwb, stride_gwd,
NUM_ROWS, D,
ROWS_TILE_SIZE: tl.constexpr, D_TILE_SIZE: tl.constexpr,
):
    row_tile_idx = tl.program_id(0)
    n_row_tiles = tl.num_programs(0)

    # Inputs
    grad_output_block_ptr = tl.make_block_ptr(
        grad_output_ptr,
        shape=(NUM_ROWS, D), strides=(stride_gr, D_TILE_SIZE),
        offsets=(row_tile_idx * ROWS_TILE_SIZE, 0),
        block_shape=(ROWS_TILE_SIZE, D_TILE_SIZE),
        order=(0, 1),
    )

    x_block_ptr = tl.make_block_ptr(
        x_ptr,
        shape=(NUM_ROWS, D), strides=(stride_xr, stride_xd),
        offsets=(row_tile_idx * ROWS_TILE_SIZE, 0),
        block_shape=(ROWS_TILE_SIZE, D_TILE_SIZE),
        order=(1, 0),
    )

    weight_block_ptr = tl.make_block_ptr(
        weight_ptr,
        shape=(D, D_TILE_SIZE), strides=(stride_wd, D_TILE_SIZE),
        offsets=(0, 0), block_shape=(D_TILE_SIZE, D_TILE_SIZE),
        order=(0, 1),
    )

    grad_x_block_ptr = tl.make_block_ptr(
        grad_x_ptr,
        shape=(NUM_ROWS, D), strides=(stride_gxr, stride_gxd),
        offsets=(row_tile_idx * ROWS_TILE_SIZE, 0),
        block_shape=(ROWS_TILE_SIZE, D_TILE_SIZE),
        order=(1, 0),
    )

    partial_grad_weight_block_ptr = tl.make_block_ptr(
        partial_grad_weight_ptr,
        shape=(n_row_tiles, D_TILE_SIZE), strides=(stride_gwb, stride_gwd),
        offsets=(row_tile_idx, 0),
        block_shape=(1, D_TILE_SIZE),
        order=(1, 0),
    )

    for i in range(tl.cdiv(D, D_TILE_SIZE)):
        grad_output = tl.load(grad_output_block_ptr, boundary_check=(0, 1), padding_option="zero") # (ROWS_TILE_SIZE, D_TILE_SIZE)

        # Outer product for grad_x
        weight = tl.load(weight_block_ptr, boundary_check=(0, 1), padding_option="zero") # (D_TILE_SIZE, D_TILE_SIZE)
        grad_x_row = grad_output[:, None] * weight[None, :]
        tl.store(grad_x_block_ptr, grad_x_row, boundary_check=(0, 1))

        # Reduce as many rows as possible for the grad_weight result
        row = tl.load(x_block_ptr, boundary_check=(0, 1), padding_option="zero") # (ROWS_TILE_SIZE, D_TILE_SIZE)
        grad_weight_row = tl.sum(row * grad_output[:, None], axis=0, keep_dims=True)
        tl.store(partial_grad_weight_block_ptr, grad_weight_row, boundary_check=(1,)) # Never out of bounds for dim 0

    # Move the pointers to the next tile along D
    x_block_ptr = x_block_ptr.advance((0, D_TILE_SIZE))
    weight_block_ptr = weight_block_ptr.advance((D_TILE_SIZE,))
    partial_grad_weight_block_ptr = partial_grad_weight_block_ptr.advance((0, D_TILE_SIZE))
    grad_x_block_ptr = grad_x_block_ptr.advance((0, D_TILE_SIZE))

```

计算梯度  $\nabla_x$  很简单，我们将结果写入输出张量的适当图块。

然而，计算  $\nabla_w$  更具挑战性。每个内核实例负责  $x$  的一行图块，但我们现在需要对  $x$  的 *across* 行求和。我们将假设 `partial_grad_weight_ptr` 包含一个  $n_{\text{row\_tiles}} \times H$  矩阵，其中第一个维度仅在  $x$  的行图块内减少，而不是直接在后向传递中进行此求和。在写入该张量之前，我们会在当前行图块内进行缩减。在内核之外，我们使用 `torch.sum` 来减少  $\nabla_w$ ，以求和每个行图块的结果。<sup>1</sup> `autograd.Function` 的最后部分相对简单：

```
class WeightedSumFunc(torch.autograd.Function):
    @staticmethod
    def forward(ctx, x, weight):
        # ... (defined earlier)

    @staticmethod
    def backward(ctx, grad_out):
        x, weight = ctx.saved_tensors
        ROWS_TILE_SIZE, D_TILE_SIZE = ctx.ROWS_TILE_SIZE, ctx.D_TILE_SIZE # These don't have to be the same
        n_rows, D = x.shape

        # Our strategy is for each thread block to first write to a partial buffer,
        # then we reduce over this buffer to get the final gradient.
        partial_grad_weight = torch.empty((triton.cdiv(n_rows, ROWS_TILE_SIZE), D), device=x.device, dtype=x.dtype)
        grad_x = torch.empty_like(x)

        weighted_sum_backward[(triton.cdiv(n_rows, ROWS_TILE_SIZE),)](
            x, weight,
            grad_out,
            grad_x, partial_grad_weight,
            x.stride(0), x.stride(1),
            weight.stride(0),
            grad_out.stride(0),
            grad_x.stride(0), grad_x.stride(1),
            partial_grad_weight.stride(0), partial_grad_weight.stride(1),
            NUM_ROWS=n_rows, D=D,
            ROWS_TILE_SIZE=ROWS_TILE_SIZE, D_TILE_SIZE=D_TILE_SIZE,
        )
        grad_weight = partial_grad_weight.sum(axis=0)
        return grad_x, grad_weight
```

最后，我们现在可以获得一个与 `torch.nn.functional` 中实现的函数非常相似的函数：

```
f_weightedsum = WeightedSumFunc.apply
```

现在，在两个 PyTorch 张量  $x$  和  $w$  上调用 `f_weightedsum` 将给出如下张量：

```
tensor([ 90.8563, -93.6815, -80.8884, ..., 103.4840, -21.4634, -24.0192],
        device='cuda:0', grad_fn=<WeightedSumFuncBackward>)
```

请注意张量上附加的 `grad_fn` - 这表明当该张量出现在计算图中时，PyTorch 知道在向后传递中调用什么。这样就完成了我们的 Triton 加权求和运算的实现。

#### 4.2.2 FlashAttention-2 前向传递

您将使用 FlashAttention-2 [T.1] 之后显着改进的 Triton 实现来替换 PyTorch 注意力实现。道，2023]。FlashAttention-2 采用一些技巧来计算分块中的前向传递，从而实现高效的内存访问模式，并避免需要在全局内存上具体化完整的注意力矩阵。

---

<sup>1</sup>Or, of course, we could write our own kernel for that.

在进入本节之前，我们强烈建议至少阅读原始的 FlashAttention 论文 [T. Dao 等人, 2022]，这将使您直观地了解利用 FlashAttention 实现高效注意力的核心技术：以跨图块的在线方式计算 softmax ([M. Milakov 等人, 2018] 中提出的技术)。我们还建议您查看 H. He [4]，以获得有关 GPU 如何实际执行 PyTorch 代码的更多直观信息。

了解普通注意力的低效率

回想一下，注意力的前向传播（暂时忽略掩蔽）可以写成：

$$S = QK^\top / \sqrt{d} \quad (4)$$

$$P_{ij} = \text{softmax}_j(S)_{ij} \quad (5)$$

$$O = PV \quad (6)$$

标准的后向传播是

$$dV = P^\top dO \quad (7)$$

$$dP = dOV^\top \quad (8)$$

$$dS_i = d\text{softmax}(dP_i) = (\text{diag}(P_i) - P_i P_i^\top) dP_i \quad (9)$$

$$dQ = dSK / \sqrt{d} \quad (10)$$

$$dK = dS^\top Q / \sqrt{d} \quad (11)$$

正如我们所看到的，后向传递依赖于前向传递的一些非常大的激活。例如，计算方程 7 中的  $dV$  需要  $P$ ，它们是形状  $(\text{batch\_size}, \text{n\_heads}, \text{seq\_len}, \text{seq\_len})$  的注意力分数 - 该激活矩阵的大小取决于 *quadratically* 序列长度，这解释了我们上面在对大序列长度的注意力进行基准测试时遇到的内存问题。在普通注意力的前向和后向传递过程中，我们付出了大量的内存 IO 成本在片上 SRAM 和 GPU HBM 之间传输  $P$  和其他大型激活。在标准实现中存在 *several* 种此类传输：例如，标准后向传递实现将在方程 7 和方程 9 的计算中从 HBM 读取  $P$ 。

FlashAttention 的主要目标是避免从 HBM 读取和写入注意力矩阵，以减少 IO 和峰值内存成本。我们使用三种技术来实现这一点：平铺、重新计算和运算符融合。

平铺

为了避免在 HBM 中读写注意力矩阵，我们在不访问整个输入的情况下计算 softmax 缩减。具体来说，我们重构注意力计算，将输入分割成图块，并对输入图块进行多次传递，从而逐步执行 softmax 缩减。

重新计算

我们避免在 HBM 中存储形状为  $(\text{batch\_size}, \text{n\_heads}, \text{seq\_len}, \text{seq\_len})$  的大型中间注意力矩阵。相反，我们将在 HBM 中保存某些“激活检查点”，然后在后向传递过程中重新计算部分前向传递，以获得计算梯度所需的其他激活。FlashAttention-2 还存储注意力分数的  $\text{logsumexp}$ ， $L$ ，它将用于简化后向传递计算。 $L$  的表达式为：

$$L_i = \log \left( \sum_j \exp(S_{ij}) \right) \quad (12)$$

在我们的最终内核中，我们将以在线方式计算它，但最终结果应该是相同的。通过平铺和重新计算，我们的内存 IO 和峰值使用不再依赖于  $\text{sequence\_length}^2$ ，因此我们可以使用更大的序列长度。

## 算子融合

最后，我们通过在单个内核中执行所有操作来避免注意力矩阵和其他中间激活的重复内存 IO，这称为算子或内核融合。我们将为前向传递编写一个 Triton 内核，该内核执行涉及注意力的所有操作，并在 HBM 和 S RAM 之间传输有限的数据。算子融合部分是通过重新计算实现的，

因为我们可以避免通常的内存 IO，所以我们需要将每个中间激活存储到 HBM。

有关这些技术的更多直观信息，请查看 FlashAttention 论文 [T. Dao 等人, 2022; T. Dao, 2023]。

## 带重新计算的向后传递

使用  $L$ ，我们可以进行适当的重新计算并有效地计算向后传递。在开始向后传递之前，我们在全局内存中预先计算值  $D = \text{rowsum}(O \circ dO)$ （其中  $\circ$  是逐元素乘法），它等于  $\text{rowsum}(P \circ dP)$ ，因为

$$PdP^\top = P(dOV^\top)^\top =$$

$(PV)dO^\top = OdO^\top$ （对于任何矩阵  $A$  和  $B$  来说，还有  $\text{rowsum}(A \circ B) = \text{diag}(AB^\top)$ ）。使用  $L$  和  $D$  向量，可以在没有 softmax 的情况下计算后向传递。向后传递的完整计算现在是：

$$S = QK^\top / \sqrt{d} \quad (13)$$

$$P_{ij} = \exp(S_{ij} - L_i) \quad (14)$$

$$dV = P^\top dO \quad (15)$$

$$dP = dOV^\top \quad (16)$$

$$dS_{ij} = P_{ij}(dP_{ij} - D_i) \quad (17)$$

$$dQ = dSK / \sqrt{d} \quad (18)$$

$$dK = dS^\top Q / \sqrt{d} \quad (19)$$

我们可以看到，操作序列并不要求我们在前向传递过程中将注意力分数  $P$  存储在 HBM 中 - 我们根据方程 13 和方程 14 中的激活  $Q$ 、 $K$  和  $L$  重新计算它们。

## FlashAttention 前向传递的详细信息

现在我们已经对 FlashAttention-2 中使用的技术有了一个高层次的了解，我们将深入了解您将实现的 FA2 前向传递内核的细节。为了避免从 HBM 读取和写入注意力矩阵，我们希望使用分片，即独立于其他分片计算输出的每个分片。这要求我们能够计算  $P$  的图块，理想情况下在两个维度上图块（对于查询和键）。

然而，当我们将 softmax 应用于  $S$  时，我们需要减少  $S$  的整行来计算 softmax 分母，这意味着我们无法直接计算图块中的  $P$ 。FlashAttention-2 使用 *online softmax* 解决了这个问题。在下面的文本中，我们将使用下标索引  $i$  表示当前查询图块，使用上标索引  $(j)$  表示当前关键图块。沿查询维度的图块的大小将为  $B_q$ ，沿关键维度的图块的大小将为  $B_k$ 。我们不会沿着隐藏维度  $d$  平铺。

我们还保留一些按行运行的值， $m_i^{(j)} \in \mathbb{R}^{B_q}$  和  $l_i^{(j)} \in \mathbb{R}^{B_q}$ 。行方向的  $m_i^{(j)}$  值是一个运行最大值，它被跟踪，以便我们可以以数值稳定的方式计算 softmax（回想一下作业 1 中的 softmax 实现中的这个技巧）。我们将用每个新行更新  $m_i^{(j)}$  -

$S$  的明智平铺（当  $j$  增加时）。使用运行最大值，我们可以将非标准化 softmax 值（分子）计算为  $\tilde{P}_i^{(j)} = \exp(S_i^{(j)} - m_i^{(j)})$ 。  $l_i^{(j)}$  是 softmax 分母的运行代理，并将使用非标准化 softmax 值作为  $l_i^{(j)} = \exp(m_i^{(j-1)} - m_i^{(j)}) \cdot l_i^{(j-1)} + \text{rowsum}(\tilde{P}_i^{(j)})$  进行更新。当我们最终写入输出时，我们需要使用  $l_i^{(T_k)}$  来完成其标准化，这是处理所有关键图块后  $l_i^{(j)}$  的最终值。算法 1 显示了 GPU 上的前向传递。

---

**Algorithm 1:** FlashAttention-2 forward pass

---

```

1 Require:  $Q \in \mathbb{R}^{N_q \times d}$ ,  $K, V \in \mathbb{R}^{N_k \times d}$ , tile sizes  $B_q, B_k$ 
2 Split  $Q$  into  $T_q = \lceil \frac{N_q}{B_q} \rceil$  tiles  $Q_1, \dots, Q_{T_q}$  of size  $B_q \times d$ 
3 Split  $K, V$  into  $T_k = \lceil \frac{N_k}{B_k} \rceil$  tiles  $K^{(1)}, \dots, K^{(T_k)}$  and  $V^{(1)}, \dots, V^{(T_k)}$  of size  $B_k \times d$ 
4 for  $i = 1, \dots, T_q$  do
5   Load  $Q_i$  from global memory
6   Initialize  $O_i^{(0)} = \mathbf{0} \in \mathbb{R}^{B_q \times d}$ ,  $l_i^{(0)} = 0 \in \mathbb{R}^{B_q}$ ,  $m_i^{(0)} = -\infty \in \mathbb{R}^{B_q}$ 
7   for  $j = 1, \dots, T_k$  do
8     Load  $K^{(j)}, V^{(j)}$  from global memory
9     Compute tile of pre-softmax attention scores  $S_i^{(j)} = \frac{Q_i (K^{(j)})^\top}{\sqrt{d}} \in \mathbb{R}^{B_q \times B_k}$ 
10    Compute  $m_i^{(j)} = \max(m_i^{(j-1)}, \text{rowmax}(S_i^{(j)})) \in \mathbb{R}^{B_q}$ 
11    Compute  $\tilde{P}_i^{(j)} = \exp(S_i^{(j)} - m_i^{(j)}) \in \mathbb{R}^{B_q \times B_k}$ 
12    Compute  $l_i^{(j)} = \exp(m_i^{(j-1)} - m_i^{(j)}) l_i^{(j-1)} + \text{rowsum}(\tilde{P}_i^{(j)}) \in \mathbb{R}^{B_q}$ 
13    Compute  $O_i^{(j)} = \text{diag}(\exp(m_i^{(j-1)} - m_i^{(j)})) O_i^{(j-1)} + \tilde{P}_i^{(j)} V^{(j)}$ 
14  end for
15  Compute  $O_i = \text{diag}(l_i^{(T_k)})^{-1} O_i^{(T_k)}$ 
16  Compute  $L_i = m_i^{(T_k)} + \log(l_i^{(T_k)})$ 
17  Write  $O_i$  to global memory as the  $i$ -th tile of  $O$ .
18  Write  $L_i$  to global memory as the  $i$ -th tile of  $L$ .
19 end for
20 Return the output  $O$  and the logsumexp  $L$ .
```

---

在我们开始在 Triton 中实现前向传播之前，我们在这里收集了一些编写 Triton 内核的一般提示和技巧。

#### Triton 提示和技巧

- 您可以在 Triton 中使用带有 `tl.device_print` 的打印语句来调试：  
[https://triton-lang.org/main/python-api/generated/triton.language.device\\_print.html](https://triton-lang.org/main/python-api/generated/triton.language.device_print.html)。  
 有一个设置 `TRITON_INTERPRET=1` 可以在 CPU 上运行 Triton 解释器，尽管我们发现它有问题。
- 定义块指针时，请确保它们具有正确的偏移量，并且块偏移量乘以适当的图块大小。
- 线程块的启动网格设置为

```
kernel_fn[(launch_grid_d1, launch_grid_d2, ...)](...arguments...)
```

在 `torch.autograd.Function` 子类的方法中，正如我们在加权和示例中看到的那样。

- 与 `tl.dot` 执行矩阵乘法。
- 要前进块指针，请使用 `*_block_ptr = *_block_ptr.advance(...)`

问题 (**flash\_forward**) : FlashAttention-2 前向传递 (15 分)

(a) 编写一个纯 PyTorch (无 Triton) `autograd.Function` 来实现 FlashAttention-2 前向传递。这将比常规 PyTorch 实现慢很多，但会帮助您调试 Triton 内核。

您的实现应采用输入  $Q$ 、 $K$  和  $V$  以及标志 `is_causal` 并生成输出  $O$  和 `logsumexp` 值  $L$ 。您可以忽略此任务的 `is_causal` 标志。然后，`autograd.Function` 前向应该保存  $L, Q, K, V, O$  用于后向传递并返回  $O$ 。请记住，`autograd.Function` 的 `forward` 方法的实现始终将上下文作为其第一个参数。任何 `autograd.Function` 类都需要实现 `backward` 方法，但现在您可以让它只引发 `NotImplementedError`。如果您需要进行比较，您可以在 PyTorch 中实现方程 4 至方程 6 和方程 12 并比较您的输出。

那么接口就是 `def forward(ctx, Q, K, V, is_causal=False)`。确定您自己的图块尺寸，但确保它们的尺寸至少为  $16 \times 16$ 。我们将始终使用 2 的幂且至少为 16 的维度来测试您的代码，因此您无需担心越界访问。

可交付成果：在前向传递中实现 FlashAttention-2 的 `torch.autograd.Function` 子类。要测试您的代码，请实施

`[adapters.get_flashattention_autograd_function_pytorch]`。然后，使用 `uv run pytest -k test_flash_forward_pass_pytorch` 运行测试并确保您的实现通过它。

(b) 按照算法 1 编写 Triton 内核，用于 FlashAttention-2 的前向传递。然后，编写  `torch.autograd.Function` 的另一个子类，在前向传递中调用此（融合）内核，而不是在 PyTorch 中计算结果。一些针对特定问题的提示：

- 为了进行调试，我们建议将您执行的每个 Triton 操作的结果与您在 (a) 部分中编写的平铺 PyTorch 实现进行比较。
- 您的启动网格应设置为  $(T_q, \text{batch\_size})$ ，这意味着每个 Triton 程序实例将仅加载单个批次索引中的元素，并且仅读取/写入  $Q$ 、 $O$  和  $L$  的单个查询图块。
- 内核应该只有一个循环，它将迭代关键图块  $1 \leq j \leq T_k$ 。
- 在循环末尾提前块指针。
- 使用下面的函数声明（使用我们给您的块指针，您应该能够推断其余指针的设置）：

```
@triton.jit
def flash_fwd_kernel(
```

```

Q_ptr, K_ptr, V_ptr,
Q_ptr, L_ptr,
stride_qb, stride_qq, stride_qd,
stride_kb, stride_kk, stride_kd,
stride_vb, stride_vk, stride_vd,
stride_ob, stride_oq, stride_od,
stride_lb, stride_lq,
N_QUERIES, N_KEYS,
scale,
D: tl.constexpr,
Q_TILE_SIZE: tl.constexpr,
K_TILE_SIZE: tl.constexpr,
):
    # Program indices
    query_tile_index = tl.program_id(0)
    batch_index = tl.program_id(1)

    # Offset each pointer with the corresponding batch index
    # multiplied with the batch stride for each tensor
    Q_block_ptr = tl.make_block_ptr(
        Q_ptr + batch_index * stride_qb,
        shape=(N_QUERIES, D),
        strides=(stride_qq, stride_qd),
        offsets=(query_tile_index * Q_TILE_SIZE, 0),
        block_shape=(Q_TILE_SIZE, D),
        order=(1, 0),
    )
    ...

```

其中,  $scale$  是  $\frac{1}{\sqrt{d}}$ ,  $Q\_TILE\_SIZE$  和  $K\_TILE\_SIZE$  分别是  $B_q$  和  $B_k$ 。您可以稍后调整这些。

这些附加指南可能会帮助您避免精度问题:

- 片上缓冲区 ( $O_i, l, m$ ) 应具有 dtype `tl.float32`。如果要累积到输出缓冲区中, 请使用 `acc` 参数 (`acc = tl.dot(..., acc=acc)`)。
- 在将它们相乘之前将  $\tilde{P}_i^{(j)}$  转换为  $V^{(j)}$  的 dtype, 并将  $O_i$  在将其写入全局内存之前将其转换为适当的 dtype。投射是通过 `tensor.to` 完成的。您可以使用 `tensor.dtype` 获取张量的 dtype, 使用 `*_block_ptr.type.element_ty` 获取块指针/指针的 dtype。

可交付成果: 使用 Triton 内核在前向传递中实现 FlashAttention-2 的 `torch.autograd.Function` 子类。实施 `[adapters.get_flash_autograd_function_triton]`。然后, 使用 `uv run pytest -k test_flash_forward_pass_triton` 运行测试并确保您的实现通过它。

(c) 添加一个标志作为您的 `autograd.Function` 实现的最后一个参数, 以进行因果屏蔽。这应该是一个布尔标志, 当设置为 `True` 时, 可以启用因果屏蔽的索引比较。您的 Triton 内核应该有一个相应的附加参数 `is_causal: tl.constexpr` (这是必需的类型注释)。在 Triton 中, 为查询和键构建适当的索引向量, 并将它们进行比较以形成大小为  $B_q \times B_k$  的方形掩码。对于被屏蔽的元素, 将常量值  $-1e6$  添加到注意力得分矩阵  $S_i^{(j)}$  的相应元素中。确保使用 `ctx.is_causal = is_causal` 保存向后的掩码标志。

可交付成果: `torch.autograd.Function` 子类的一个附加标志, 它使用 Triton 内核实现带有因果屏蔽的 FlashAttention-2 前向传递。确保该标志是可选的并且默认为 `False`, 以便前面的测试仍然通过。

通过重新计算实现向后传递

请注意，与方程 7到方程 11中的标准向后传递不同，我们可以使用重新计算来避免方程 13到方程 19中所示的向后传递中的softmax操作。这意味着我们可以使用一个简单的内核来计算向后传递，并且不需要在线技巧。因此，对于这一部分，您可以通过在常规 PyTorch 函数（不是 Triton）上调用 `torch.compile` 来实现向后。

问题 (**flash\_backward**): FlashAttention-2 向后传递 (5 分)

使用 PyTorch（不是 Triton）和 `torch.compile` 实现 FlashAttention-2 `autograd.Function` 的向后传递。您的实现应将  $Q$ 、 $K$ 、 $V$ 、 $O$ 、 $dO$  和  $L$  张量作为输入，并返回  $dQ$ 、 $dK$  和  $dV$ 。请记住计算并使用  $D$  向量。您可以按照方程 13 至方程 19 进行计算。

可交付成果：要测试您的实现，请运行 `uv run pytest -k test_flash_backward`。

现在，我们将 FlashAttention-2 的（部分）Triton 实现与常规 Attention 的 PyTorch 实现的性能进行比较。

问题 (**flash\_benchmarking**): FlashAttention-2 基准测试 (5 分)

(a) 使用 `triton.testing.do_bench` 编写一个基准测试脚本，将 FlashAttention-2 前向和后向传递的（部分）Triton 实现与常规 PyTorch 实现（即不使用 FlashAttention）的性能进行比较。

具体来说，您将报告一个表格，其中包括 Triton 和 PyTorch 实现的前向、后向和端到端前向-后向传递的延迟。在开始基准测试之前随机生成任何必要的输入，并在单个 B200 上运行基准测试。始终使用批量大小 1 和因果掩蔽。扫描从 128 到 65536 的各种 2 的幂序列长度的笛卡尔积，嵌入从 16 到 128 的各种 2 的幂的维度大小，以及 `torch.bfloat16` 和 `torch.float32` 的精度。您可能需要根据输入大小调整图块大小。

可交付成果：使用上述设置并报告前向、后向和端到端延迟，将 FlashAttention-2 实现与 PyTorch 实现进行比较的结果表。

#### 4.2.3 可选：Triton 向后传递

如果您有兴趣更多地练习 Triton 和/或快速提交排行榜，我们提供了平铺的 FlashAttention-2 向后传递，您可以在 Triton 中实现下面的功能。算法 2 显示了 FlashAttention-2 向后传递，因为它应该在 Triton 中实现。这里的一个关键技巧是计算  $P$  两次，一次计算  $dQ$ ，另一次计算  $dK$  和  $dV$ 。这让我们可以跳过线程块之间的同步，这意味着我们可以避免缓慢的原子操作。

---

**Algorithm 2:** Tiled FlashAttention-2 backward pass

---

```
1 Require:  $Q, O, dO \in \mathbb{R}^{N_q \times d}$ ,  $K, V \in \mathbb{R}^{N_k \times d}$ ,  $L \in \mathbb{R}^{N_q}$ , tile sizes  $B_q, B_k$ 
2 Compute  $D = \text{rowsum}(O \circ dO) \in \mathbb{R}^{N_q}$ 
3 Split  $Q, O, dO$  into  $T_q = \lceil \frac{N_q}{B_q} \rceil$  tiles  $Q_1, \dots, Q_{T_q}, O_1, \dots, O_{T_q}, dO_1, \dots, dO_{T_q}$ , each of size  $B_q \times d$ 
4 Split  $K, V$  into  $T_k = \lceil \frac{N_k}{B_k} \rceil$  tiles  $K^{(1)}, \dots, K^{(T_k)}$  and  $V^{(1)}, \dots, V^{(T_k)}$ , each of size  $B_k \times d$ 
5 Split  $L, D$  into  $T_q$  tiles  $L_1, \dots, L_{T_q}$  and  $D_1, \dots, D_{T_q}$ , each of size  $B_q$ 
6 for  $j = 1, \dots, T_k$  do
7   Load  $K^{(j)}, V^{(j)}$  from global memory
8   Initialize  $dK_0^{(j)} = dV_0^{(j)} = \mathbf{0} \in \mathbb{R}^{B_k \times d}$ 
9   for  $i = 1, \dots, T_q$  do
10    Load  $Q_i, dO_i$  from global memory
11    Compute tile of attention scores  $S_i^{(j)} = \frac{Q_i(K^{(j)})^\top}{\sqrt{d}} \in \mathbb{R}^{B_q \times B_k}$ 
12    Compute attention probabilities  $P_i^{(j)} = \exp(S_i^{(j)} - L_i) \in \mathbb{R}^{B_q \times B_k}$ 
13    Compute  $dV_i^{(j)} = dV_{i-1}^{(j)} + (P_i^{(j)})^\top dO_i \in \mathbb{R}^{B_k \times d}$ 
14    Compute  $dP_i^{(j)} = dO_i (V^{(j)})^\top \in \mathbb{R}^{B_q \times B_k}$ 
15    Compute  $dS_i^{(j)} = P_i^{(j)} \circ (dP_i^{(j)} - D_i) \in \mathbb{R}^{B_q \times B_k}$ 
16    Compute  $dK_i^{(j)} = dK_{i-1}^{(j)} + (dS_i^{(j)})^\top Q_i / \sqrt{d} \in \mathbb{R}^{B_k \times d}$ .
17  end for
18  Write  $dK_{T_q}^{(j)}$  and  $dV_{T_q}^{(j)}$  to global memory as the  $j$ -th tiles of  $dK$  and  $dV$ .
19 end for
20 for  $i = 1, \dots, T_q$  do
21   Load  $Q_i, dO_i$  from global memory
22   Initialize  $dQ_i^{(0)} = \mathbf{0} \in \mathbb{R}^{B_q \times d}$ 
23   for  $j = 1, \dots, T_k$  do
24    Load  $K^{(j)}, V^{(j)}$  from global memory
25    Compute tile of attention scores  $S_i^{(j)} = \frac{Q_i(K^{(j)})^\top}{\sqrt{d}} \in \mathbb{R}^{B_q \times B_k}$ 
26    Compute attention probabilities  $P_i^{(j)} = \exp(S_i^{(j)} - L_i) \in \mathbb{R}^{B_q \times B_k}$ 
27    Compute  $dP_i^{(j)} = dO_i (V^{(j)})^\top \in \mathbb{R}^{B_q \times B_k}$ 
28    Compute  $dS_i^{(j)} = P_i^{(j)} \circ (dP_i^{(j)} - D_i) \in \mathbb{R}^{B_q \times B_k}$ 
29    Compute  $dQ_i^{(j)} = dQ_i^{(j-1)} + dS_i^{(j)} K^{(j)} / \sqrt{d} \in \mathbb{R}^{B_q \times d}$ 
30  end for
31  Write  $dQ_i^{(T_k)}$  to global memory as the  $i$ -th tile of  $dQ$ .
32 end for
33 Return  $dQ, dK, dV$ .
```

---

## 5 分布式数据并行训练

在作业的下一部分中，我们将探索使用多个 GPU 训练语言模型的方法，重点关注数据并行性。我们将从 PyTorch 中的分布式通信入门开始。然后，我们将研究分布式数据并行训练的简单实现，然后实现通信效率的各种改进并进行基准测试。

## 5.1 PyTorch 中的单节点分布式通信

让我们首先看一下 PyTorch 中的一个简单的分布式应用程序，其目标是生成四个随机整数张量并计算它们的总和。

在下面的分布式情况下，我们将生成四个工作进程，每个进程都会生成一个随机整数张量。为了对工作进程中的这些张量求和，我们将调用全归约集体通信操作，它将每个进程上的原始数据张量替换为全归约结果（即总和）。

现在让我们看一些代码。

```
import os
import torch
import torch.distributed as dist
import torch.multiprocessing as mp

def setup(rank, world_size):
    os.environ["MASTER_ADDR"] = "localhost"
    os.environ["MASTER_PORT"] = "29500"
    dist.init_process_group("gloo", rank=rank, world_size=world_size)

def distributed_demo(rank, world_size):
    setup(rank, world_size)
    data = torch.randint(0, 10, (3,))
    print(f"rank {rank} data (before all-reduce): {data}")
    dist.all_reduce(data, async_op=False)
    print(f"rank {rank} data (after all-reduce): {data}")

if __name__ == "__main__":
    world_size = 4
    mp.spawn(fn=distributed_demo, args=(world_size, ), nprocs=world_size, join=True)
```

运行上面的脚本后，我们得到下面的输出。正如预期的那样，每个工作进程最初持有不同的 `data` 张量。在对所有工作进程的张量求和的 `all-reduce` 操作之后，在每个工作进程上就地修改 `data` 以保存 `all-reduce` 结果。<sup>2</sup>

```
$ uv run python distributed_hello_world.py
rank 3 data (before all-reduce): tensor([3, 7, 8])
rank 0 data (before all-reduce): tensor([4, 4, 7])
rank 2 data (before all-reduce): tensor([6, 0, 7])
rank 1 data (before all-reduce): tensor([9, 5, 3])
rank 1 data (after all-reduce): tensor([22, 16, 25])
rank 0 data (after all-reduce): tensor([22, 16, 25])
rank 3 data (after all-reduce): tensor([22, 16, 25])
rank 2 data (after all-reduce): tensor([22, 16, 25])
```

现在让我们更仔细地回顾一下上面的脚本。命令 `mp.spawn` 会生成 `nprocs` 进程，这些进程使用提供的 `args` 运行 `fn`。另外，函数 `fn` 被称为 `fn(rank, *args)`，其中 `rank` 是工作进程的索引（0到 `nprocs-1` 之间的值）。因此，我们的 `distributed_demo` 函数

---

<sup>2</sup>If you run this script multiple times, you'll notice that the order of the printed output is not deterministic. Since this application is running in a distributed setting, we cannot control the exact order in which commands are being run—our only guarantee is that after the `all-reduce` operation is complete, the separate processes will hold bitwise identical result tensors.

必须接受这个整数等级作为其第一个位置参数。另外，我们传入`world_size`，它指的是工作进程总数。

每个工作进程都属于一个 *process group*，它是通过 `dist.init_process_group` 初始化的。进程组代表多个工作进程，它们将通过共享主进程进行协调和通信。主站由其 IP 地址和端口定义，主站运行等级为 0 的进程。像 all-reduce 这样的集体通信操作对进程组中的每个进程进行操作。

在本例中，我们使用 "gloo" 后端初始化进程组，但其他后端也可用。特别是，"nccl" 后端将使用 NVIDIA NCCL 集体通信库，该库通常对于 CUDA 张量来说性能更高。但是，NCCL 只能在具有 GPU 的计算机上使用，而 Gloo 可以在仅具有 CPU 的计算机上运行。您应该始终使用 NCCL 进行分布式 GPU 训练，并且仅在没有可用 GPU 的情况下使用 Gloo 进行本地开发。我们在此示例中使用 Gloo，因为它可以在仅使用 CPU 的计算机上进行本地执行和开发。

运行多GPU作业时，请确保不同的Rank使用不同的GPU。一种方法是在 `setup` 函数中调用 `torch.cuda.set_device(rank)`，这样 `tensor.to("cuda")` 就会自动将其移动到指定的设备。或者，您可以显式创建每个级别的设备字符串（例如，`device = f"cuda:{rank}"`），然后使用该设备字符串作为任何数据移动的目标设备（例如，`tensor.to(f"cuda:{rank}")`）。

## 术语

在作业的其余部分（以及您可能在网上看到的各种其他资源），您可能在 PyTorch 分布式通信的上下文中遇到以下术语。尽管我们在本次作业中将重点关注单节点、多进程分布式训练，但该术语对于总体理解分布式训练很有用。请参阅图 3 的视觉表示。



**node**网络上的一台机器。

**worker**参与分布式训练的程序实例。在此分配中，每个工作线程将有一个进程，因此我们将互换使用 *worker*、*process* 和 *worker process*。然而，工作人员可能会使用多个进程（例如，加载数据进行训练），因此这些术语在实践中并不总是等效的。

**world size**进程组中的工人总数。

**global rank**一个整数 ID（介于 0 和 `world_size-1` 之间），唯一标识进程组中的工作线程。例如，对于世界大小为 2 的情况，一个进程的全局排名为 0（主进程），另一个进程的排名为 1。

**local world size**当跨不同节点运行应用程序时，本地世界大小是在给定节点上本地运行的工作人员数量。例如，如果我们有一个应用程序，每个应用程序在 2 个节点上生成 4 个工作线程，则世界大小将为 8，本地世界大小将为 4。请注意，在单个节点上运行时，工作线程的本地世界大小等于（全局）世界大小。

**local rank**一个整数ID（0到`local_world_size-1`之间），唯一标识机器上本地worker的索引。例如，如果我们有一个应用程序，每个进程在 2 个节点上生成 4 个进程，则每个节点将具有本地等级为 0、1、2 和 3 的工作人员。请注意，运行单节点多进程分布式应用程序时，进程的本地等级相当于其全局等级。

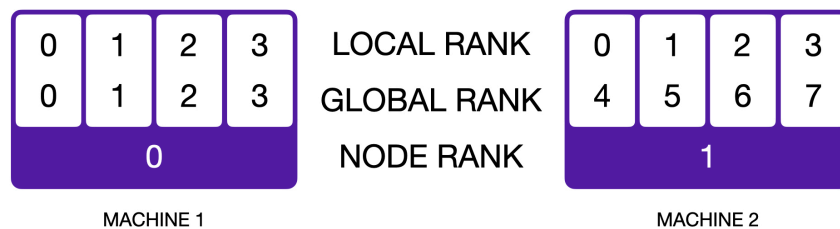


图 3: 在世界大小为 8 的 2 个节点上运行的分布式应用程序的示意图。每个工作进程由全局排名（从 0 到 7）和本地排名（从 0 到 3）标识。图取自

[https://lightning.ai/docs/fabric/stable/advanced/distributed\\_communication.html](https://lightning.ai/docs/fabric/stable/advanced/distributed_communication.html)

### 5.1.1 分布式应用程序基准测试的最佳实践

在作业的这一部分中，您将对分布式应用程序进行基准测试，以更好地了解通信的开销。以下是一些最佳实践：

- 只要有可能，就在同一台机器上运行基准测试，以便于进行受控比较。
- 在计时感兴趣的操作之前执行几个预热步骤。这对于 NCCL 通信呼叫尤其重要。5 次预热迭代通常就足够了。
- 在 GPU 上进行基准测试时，调用 `torch.cuda.synchronize()` 等待 CUDA 操作完成。请注意，即使使用 `async_op=False` 调用通信操作也是必要的，当操作在 GPU 上排队时返回（而不是通信实际完成时）。<sup>3</sup>
- 不同级别的时间可能略有不同，因此通常会汇总跨级别的测量结果以改进估计。您可能会发现 `all-gather` 集合（特别是 `dist.all_gather_object` 函数）对于收集所有等级的结果非常有用。
- 一般来说，在 CPU 上使用 Gloo 进行本地调试，然后根据给定问题的需要，在 GPU 上使用 NCCL 进行基准测试。

后端之间的切换应该像修改 `init_process_group` 调用和张量设备转换一样简单。

问题（**`distributed_communication_single_node`**）： 分布式通信（单节点）（5分）

编写一个脚本来对单节点多进程设置中的 `all-reduce` 操作的运行时间进行基准测试。上面的示例代码可以提供一个合理的起点。尝试改变以下设置：

**all-reduce data size** float32 数据张量范围超过 1MB、10MB、100MB、1GB。

**Number of GPUs/processes** 2、4 或 6。

资源要求：最多 6 个 GPU。每次基准测试运行时间不应超过 5 分钟。

可交付成果：比较各种设置的图和/或表格，用 2-3 句话评论您的结果以及对各种因素如何相互作用的想法。

<sup>3</sup>See <https://github.com/pytorch/pytorch/issues/68112#issuecomment-965932386> for more details.

## 5.2 分布式数据并行训练的简单实现

现在我们已经了解了在 PyTorch 中编写分布式应用程序的基础知识，让我们构建分布式数据并行 (DDP) 训练的最小实现。

数据并行性将批次拆分到多个设备（例如 GPU）上，从而能够对不适合单个设备的大批量进行训练。例如，假设有四个设备，每个设备可以处理最大批量大小为 32，则数据并行训练将实现有效批量大小为 128。

以下是简单地进行分布式数据并行训练的步骤。最初，每个设备都会构建一个（随机初始化）模型。我们使用广播集体通信操作将模型参数从等级 0 发送到所有其他等级。在训练开始时，每个设备都保存模型参数和优化器状态的相同副本（例如 Adam 中累积的梯度统计数据）。

1. 给定一个包含  $n$  个示例的批次，该批次被分片，并且每个设备接收  $n/d$  个不相交的示例（其中  $d$  是用于数据并行训练的设备数量）。 $d$  应该除以  $n$ ，否则某些等级会比其他等级做更多的工作，并且该步骤会受到最慢的瓶颈。
2. 每个设备使用其模型参数的本地副本对其  $n/d$  示例运行前向传递，并运行后向传递以计算梯度。请注意，此时，每个设备都保存根据其收到的  $n/d$  示例计算出的梯度。
3. 然后，我们使用 all-reduce 集体通信操作来平均不同设备上的梯度，因此每个设备都保存所有  $n$  示例的平均梯度。
4. 接下来，每个设备运行优化器步骤来更新其参数副本 - 从优化器的角度来看，它只是优化本地模型。参数和优化器状态将在所有不同设备上保持同步，因为它们都从相同的初始模型和优化器状态开始，并为每次迭代使用相同的平均梯度。至此，我们已经完成了一次训练迭代，可以重复该过程。

### 问题 (naive\_ddp): 朴素 DDP (5 分)

可交付成果：实现一种简单形式的分布式数据并行训练，在向后传递后全部减少单个参数梯度。要测试您的实现，请实现 `adapters.get_ddp` 和（可选）`adapters.ddp_on_after_backward`，然后运行 `uv run pytest tests/test_ddp.py`。

### 问题 (naive\_ddp\_benchmarking): Naïve DDP 基准测试 (3 分)

在这个简单的 DDP 实现中，每次向后传递后，参数梯度都会在各个等级上单独全部减少。为了更好地了解数据并行训练的开销，请创建一个脚本，以在使用这种简单的 DDP 实现进行训练时对之前实现的语言模型进行基准测试。测量每个训练步骤的总时间以及沟通梯度所花费的时间比例。在单节点设置（1 个节点 x 2 个 GPU）中收集第 2.1.2 节中所述的 xl 模型大小的测量结果。

可交付成果：对基准测试设置的描述，以及每次训练迭代的测量时间以及为每个设置传达梯度所花费的时间。

## 5.3 最小 DDP 实施的改进

我们在第 5.2 节中看到的最小 DDP 实现有几个关键限制：

1. 它对每个参数张量进行单独的全归约操作。每个通信调用都会产生开销，因此批量通信调用以最小化此开销可能是有利的。
2. 在传递梯度之前等待向后传递完成。然而，向后传递是增量计算的。因此，当参数梯度准备好时，可以立即进行通信，而无需等待其他参数的梯度。这使我们能够将梯度通信与后向传递的计算重叠，从而减少分布式数据并行训练的开销。

在作业的这一部分中，我们将依次解决这些限制并衡量对训练速度的影响。

### 5.3.1 减少通信调用次数

让我们看看是否可以通过批处理 all-reduce 来提高性能，而不是为每个参数张量发出通信调用。具体来说，我们将采用想要全部归约的梯度，将它们连接成一个张量，然后对所有等级的组合梯度进行全部归约。使用 `torch.utils.flatten_dense_tensors` 和 `torch.utils.unflatten_dense_tensors` 可能会有所帮助。

问题 (`minimal_ddp_flat_benchmarking`): 具有平坦梯度基准的最小 DDP (2 分)

修改您的最小 DDP 实现，以传达具有所有参数的平坦梯度的张量。将其性能与最小 DDP 实现进行比较，该实现在先前使用的条件下（1 节点 x 2 GPU，xl 模型大小，如第 2.1.2 节中所述）为每个参数张量发出全归约。

可交付成果：每次训练迭代的测量时间以及在分布式数据并行训练下通过单个批量 all-reduce 调用传达梯度所花费的时间。1-2 句话比较批处理与单独通信梯度时的结果。

### 5.3.2 与各个参数梯度通信的重叠计算

虽然批处理通信调用可能有助于降低与发出大量小型全归约操作相关的开销，但所有通信时间仍然直接导致开销。为了解决这个问题，我们可以利用后向传递增量计算每一层梯度的观察结果（从损失开始并向输入移动），因此，我们可以在参数梯度准备好后立即减少参数梯度，通过重叠计算后向传递和梯度通信来减少数据并行训练的开销。

我们将首先实现一个分布式数据并行包装器并对其进行基准测试，该包装器在单个参数张量在向后传递过程中准备好时异步地全部归约。以下提示可能有用：

后向挂钩 要在后向传递中累积梯度后自动对参数调用函数，可以使用 `register_post_accumulate_grad_hook` 函数。<sup>4</sup>

<sup>4</sup>See [https://pytorch.org/docs/stable/generated/torch.Tensor.register\\_post\\_accumulate\\_grad\\_hook.html](https://pytorch.org/docs/stable/generated/torch.Tensor.register_post_accumulate_grad_hook.html) for more information and usage examples.

异步通信 所有 PyTorch 集体通信操作都支持同步 (`async_op=False`) 和异步执行 (`async_op=True`)。同步调用将阻塞，直到集体操作在 GPU 上排队。这并不意味着 CUDA 操作已完成，因为 CUDA 操作是异步的。话虽这么说，后来使用输出的函数调用将按预期运行。<sup>5</sup> 相反，异步调用将返回分布式请求句柄 - 因此，当函数返回时，不能保证集体通信操作已在 GPU 上排队，更不用说完成了。要等待操作在 GPU 上排队（从而使输出可在以后的操作中使用），您可以在返回的通信句柄上调用 `handle.wait()`。

例如，以下两个示例通过同步或异步调用对张量列表中的每个张量进行全归约：

```
tensors = [torch.rand(5) for _ in range(10)]

# Synchronous, block until operation is queued on the GPU.
for tensor in tensors:
    dist.all_reduce(tensor, async_op=False)

# Asynchronous, return immediately after each call and
# wait on results at the end.
handles = []
for tensor in tensors:
    handle = dist.all_reduce(tensor, async_op=True)
    handles.append(handle)

# ...
# Possibly execute other commands that don't rely on the all_reduce results
# ...

# Ensure that all-reduce calls were queued and
# therefore other operations depending on the
# all-reduce output can be queued.
for handle in handles:
    handle.wait()
handles.clear()
```

问题 (`ddp_overlap_individual_parameters`): 具有重叠的各个参数的 DDP (5 分)

实现一个 Python 类来处理分布式数据并行训练。该类应该包装任意 PyTorch `nn.Module` 并负责在训练之前广播权重（因此所有等级都具有相同的初始参数）并发出梯度平均的通信调用。我们推荐以下公共接口：

**def \_\_init\_\_(self, module: torch.nn.Module):** 给定一个要并行化的实例化 PyTorch `nn.Module`，构建一个 DDP 容器来处理跨等级的梯度同步。

**def forward(self, \*inputs, \*\*kwargs):** 使用提供的位置和关键字参数调用包装模块的 `forward()` 方法。

<sup>5</sup>In advanced cases, if you are using multiple CUDA streams, you may need explicit synchronization across streams to ensure that the output is ready for later operations. See <https://pytorch.org/docs/stable/notes/cuda.html#cuda-streams>.

**def finish\_gradient\_synchronization(self):**调用时，等待异步通信调用在 GPU 上完成。

要使用此类执行分布式训练，我们将向其传递一个要包装的模块，然后在运行 `optimizer.step()` 之前添加对 `finish_gradient_synchronization()` 的调用，以确保优化器步骤（依赖于梯度的操作）可以安全地排队：

```
model = ToyModel().to(device)
ddp_model = DDP(model)

for _ in range(train_steps):
    x, y = get_batch()
    logits = ddp_model(x)
    loss = loss_fn(logits, y)
    loss.backward()
    ddp_model.finish_gradient_synchronization()
    optimizer.step()
```

可交付成果：实现一个容器类来处理分布式数据并行训练。这个类应该与梯度通信和反向传递的计算重叠。要测试您的 DDP 类，首先实现适配器 `[adapters.get_ddp]` 和 `[adapters.ddp_on_after_backward]`（后者是可选的，根据您的实现，您可能不需要它）。

然后，要执行测试，请运行 `uv run pytest tests/test_ddp.py`。我们建议多次运行测试（例如 5 次）以确保其可靠通过。

**问题 (ddp\_overlap\_individual\_parameters\_benchmarking):** DDP 重叠单个参数基准测试（1 分）

(a) 当向后传递计算与各个参数梯度的通信重叠时，对 DDP 实现的性能进行基准测试。将其性能与我们之前研究的设置（最小 DDP 实现，要么为每个参数张量发出全归约，要么对所有参数张量的串联发出单个全归约）进行比较，设置相同：1 个节点、2 个 GPU 和第 2.1.2 节中描述的 xl 模型大小。



可交付成果：将反向传递与各个参数梯度的通信重叠时每次训练迭代的测量时间，并用 1-2 句话比较结果。

(b) 使用 Nsight 分析器检测基准测试代码（使用 1 个节点、2 个 GPU、xl 模型大小设置），将初始 DDP 实现与此重叠实现进行比较。直观地比较这两个跟踪，并提供分析器屏幕截图，证明一种实现将计算与通信重叠，而另一种则没有。

可交付成果：两张屏幕截图（一张来自最初的 DDP 实现，另一张来自此 DDP 实现，其中计算与通信重叠），直观地显示通信是否与后向传递重叠。

## 6 优化器状态分片

分布式数据并行训练在概念上很简单并且通常非常有效，但要求每个等级保存模型参数和优化器状态的不同副本。这种冗余可能会带来巨大的内存成本。例如，AdamW 优化器为每个参数维护两个浮点数，这意味着它消耗的内存是模型权重的两倍。S. Rajbhandari 等人。[5] 描述了几种减少数据并行训练中冗余的方法，方法是跨等级划分 (1) 优化器状态、(2) 梯度和 (3) 参数，并根据需要在工作人员之间进行通信。

在作业的这一部分中，我们将通过实现简化版本的优化器状态分片来减少每列的内存消耗。每个等级的优化器实例不会保留所有参数的优化器状态，而是仅处理参数的子集（大约  $1/\text{world\_size}$ ）。当每个等级的优化器采取优化器步骤时，它只会更新其分片中的模型参数子集。然后，每个等级将其更新的参数广播到其他等级，以确保模型参数在每个优化器步骤之后保持同步。

问题 (`optimizer_state_sharding`)： 优化器状态分片 (15分)

实现一个 Python 类来处理优化器状态分片。该类应该包装任意输入 PyTorch `optim.Optimizer` 并负责在每个优化器步骤之后同步更新的参数。我们推荐以下公共接口：

**def \_\_init\_\_(self, params, optimizer\_cls: Type[Optimizer], \*\*kwargs):** 初始化分片状态优化器。params 是要优化的参数的集合（或参数组，如果用户想要对模型的不同部分使用不同的超参数，例如学习率）；这些参数将在所有级别上进行分片。optimizer\_cls 参数指定要包装的优化器的类型（例如，`optim.AdamW`）。最后，任何剩余的键字参数都被转发到 optimizer\_cls 的构造函数。确保在此方法中调用 `torch.optim.Optimizer` 超类构造函数。

**def step(self, closure, \*\*kwargs):** 使用提供的闭包和键字参数调用包装的优化器的 `step()` 方法。更新参数后，与其他队伍同步。

**def add\_param\_group(self, param\_group: dict[str, Any]):** 此方法应该向分片优化器添加一个参数组。这是在超类构造函数构建分片优化器期间调用的，也可以在训练期间调用（例如，用于逐渐解冻模型中的层）。因此，此方法应该处理在等级之间分配模型的参数。

可交付成果：实现一个容器类来处理优化器状态分片。要测试分片优化器，首先实现适配器 [`adapters.get_sharded_optimizer`]。然后，要执行测试，请运行 `uv run pytest tests/test_sharded_optimizer.py`。我们建议多次运行测试（例如 5 次）以确保它们可靠地通过。

现在我们已经实现了优化器状态分片，让我们分析一下它对训练期间内存使用峰值的影响以及运行时开销。

问题 (**optimizer\_state\_sharding\_accounting**) : 优化器状态分片核算 (5分)

(a) 创建一个脚本来分析使用和不使用优化器状态分片训练语言模型时的峰值内存使用情况。使用标准配置 (1 个节点、2 个 GPU、xl 模型大小), 报告模型初始化后、优化器步骤之前和优化器步骤之后的峰值内存使用情况。结果符合您的预期吗? 细分每个设置中的内存使用情况 (例如, 参数的内存量、优化器状态的内存量等)。

可交付成果: 2-3 句话的响应, 其中包含峰值内存使用结果以及如何在不同模型和优化器组件之间分配内存的详细信息。

(b) 我们的优化器状态分片的实现如何影响训练速度? 测量标准配置 (1 个节点、2 个 GPU、XL 模型大小) 在有和没有优化器状态分片的情况下每次迭代所花费的时间。

可交付成果: 根据您的时间安排做出 2-3 句话回应。

(c) 我们的优化器状态分片方法与 ZeRO 阶段 1 (在 S. Rajbhandari、J. Rasley、O. Ruwase 和 Y. He [5] 中描述为 ZeRO-DP  $P_{os}$ ) 有何不同?

可交付成果: 用 2-3 句话总结任何差异, 尤其是与记忆和流量相关的差异。

## 7 全分片数据并行

通过优化器状态分片和数据并行, 我们能够在数据并行轴上分割优化器状态和激活。然而, 我们的模型权重仍然是重复的——我们在每个 GPU 上存储它们的完整副本。

我们可以通过将数据并行 (DP) 轴转变为完全分片的数据并行轴 (FSDP) 来解决这个问题。使用 FSDP, 每个 GPU 仅存储每个权重张量的自己的切片, 但必须使用全收集从其他 GPU 中提取切片以形成完整的权重张量, 为前向或后向传递做好准备。

为了避免 GPU 计算等待通信完成, 大多数 FSDP 实现都会在操作之前安排层的全部收集, 这意味着相关权重在需要之前就已准备好, 从而防止通信阻塞计算。这使得权重分片通信远离关键路径, 这意味着只要通信能够跟上计算的速度并且调度得当, 它就没有成本。

有些层的内存和计算量足够小, 因此传输的延迟开销不值得。您应该将这些层标记为不被 FSDP 分片。在我们的架构中, 这主要是规范的情况。这给我们留下了嵌入层和每个线性层。

虽然在 FP32 中需要掌握权重 *store* (任何重复累加的值都对精度敏感), 但在 FP32 中权重不需要是 *used*。在混合精度中, 我们总是在使用之前转换为低精度计算数据类型, 因此我们甚至可以在传递权重之前进行转换以节省带宽。

问题 (**fsdp**) : 全分片数据并行 (15分)

实现用于完全分片数据并行训练的 Python 类。该类应该包装任意 PyTorch `nn.Module`（您的完整模型），并挂钩或包装其中的任何 `Linear` 或 `Embedding` 层。我们推荐以下公共接口：

```
def __init__(self, module: torch.nn.Module, compute_dtype: torch.dtype | None = None):
```

给定一个要并行化的实例化 PyTorch `nn.Module`，构建一个 FSDP 模块来处理权重全收集和梯度减少分散。确保您的钩子或模块包装器及时收集所有重量以进行向前传递。为了限制内存使用，仅在当前层完成前向传递之前的第二层之后开始收集。在向后传递中，您的钩子或模块包装器应该全部聚集以获得可用于计算的权重。当梯度可用时，应将它们减少分散到适当的等级。使用后请务必释放聚集的重物。当提供 `compute_dtype` 时，在通信或使用权重进行计算之前将权重转换为该数据类型，同时在 FP32 中保留主权重和优化器更新。

```
def forward(self, *inputs, **kwargs):
```

使用提供的位置和关键字参数调用包装模块的 `forward()` 方法。

```
def finish_gradient_synchronization(self):
```

调用时，等待异步通信调用在 GPU 上完成。

可交付成果：实现一个容器类来处理完全分片的数据并行训练。该容器的每个分片都应兼容作业 1 中的标准 AdamW 实现。要测试您的 FSDP 实现，请实现适配器 `[adapters.get_fsdp]`。使用 `uv run pytest tests/test_fsdp.py` 运行测试。我们建议多次运行测试（例如 5 次）以捕获任何竞争条件。

问题 (**fsdp\_accounting**): FSDP 会计 (5 分)

(a) 根据第 6 节中的分析，您预计通过实施 FSDP 可以从峰值中节省多少内存？您可以忽略在计算中将权重全部收集到每个 GPU 所需的预分配缓冲区的大小。

可交付成果：用 2-3 句话回答您的发现。

(b) 在两个 GPU 上分析 `xl` 模型并注意权重的全聚集。通信是否能及时完成前向传递？

可交付成果：根据您的时间安排做出 2-3 句话回应。附上 Nsight 的屏幕截图来支持您的主张。

## 8 分析并行策略

我们可以沿着更多的轴并行化我们的训练过程。一些常见的策略包括：

- 数据并行性 (DP) — 批量数据分布在多个设备上，每个设备计算自己批次的梯度。然后对各个设备的梯度进行平均。
- 全分片数据并行 (FSDP) — 除了数据并行之外，我们还跨设备分割优化器状态、梯度和权重，以减少内存使用。然后，设备需要在向前和向后传递期间从其他设备收集权重碎片。

- 张量并行性 (TP) — 权重矩阵在输入或输出维度上进行分片。设备计算与其分片相对应的激活，然后跨设备减少或收集激活。
- 管道并行性 (PP) — 模型被分层分成多个阶段，其中每个阶段是在不同的设备上运行。
- 专家并行 (EP) — 专家混合模型中的专家被分配到不同的设备上，每个设备为自己的专家计算输出。

在本节中，我们将在简化的设置中进行一些基本数学运算，以在并行策略之间进行选择并决定如何组合它们。首先，我们将重点关注 DP、FSDP、TP 及其组合。我们的方法是计算每个策略的通信成本，并将其与计算成本进行比较，这告诉我们在通信成本成为瓶颈之前我们可以扩展到多少设备。

要更详细地处理 TPU/GPU 拓扑和并行策略，TPU Scaling Book (J. Austin *et al.* [6]) 是一个很好的资源。有关更详细的管道并行讨论，请参阅超大规模剧本附录 (H. Z. P. N. M. M. L. W. T. W. Nouamane Tazi Ferdinand Mom [7])。这些书籍的其余部分还包含许多您可能会觉得有用的其他信息。

## 8.1 通信原语

我们的第一步是了解通信原语。在我们的简化设置中，假设我们有编号为  $0, \dots, N-1$  的  $N$  设备，并且每对设备都通过链路连接。我们还假设每个设备都有  $W$  出口（即传出）带宽；换句话说，每个设备可以以每秒  $W$  字节的速率向另一个设备发送数据。我们如何实现聚集和减少？

实现全收集操作的一种常见方法是环全收集。回想一下，在全收集集中，每个设备  $i$  以大小为  $\frac{S}{N}$  的块  $x_i$  开始，并以大小为  $S$ （以字节为单位）的整个  $x = [x_0, \dots, x_{N-1}]$  结束。在全聚集环中，我们将设备排列成一个圆圈。在每个步骤中，每个设备将其当前块发送到其右侧的下一个设备，并存储从其左侧的设备接收到的块。重复此过程，每个设备将刚刚接收到的块传递到右侧，并从左侧接收新的块。经过  $N-1$  步骤后，每个设备都拥有完整的张量。

在我们的理想化设置中，每个设备在每个步骤中同时传输大小为  $\frac{S}{N}$  的块，出口带宽为  $W$ ，并且有  $N-1$  个步骤，因此环全部收集需要  $\frac{N-1}{N} \frac{S}{W}$  秒。

接下来我们来分析环减散。在 reduce-scatter 中，每个设备  $i$  以大小为  $S$  的完整张量  $x^{(i)}$  开始。然后我们想要计算减少量  $y = \sum_{i=0}^{N-1} x^{(i)}$ ，但每个设备  $i$  最终只有一个大小为  $\frac{S}{N}$  的块  $y_i$ 。就像环聚集一样，我们首先将设备排列成一个圆圈。每个设备首先将其张量  $x^{(i)}$  划分为  $N$  块  $[x_0^{(i)}, \dots, x_{N-1}^{(i)}]$ ，每个块的大小为  $\frac{S}{N}$ 。然后我们将通过

块周围就像环全聚集一样，除了在传递块之前，每个设备将其贡献添加到块（存储部分总和）。具体来说：

对于步骤  $t = 1, \dots, N-1$ ，设备  $i$  执行以下操作：

- 如果是  $t = 1$ ，则初始化  $y \leftarrow x^{(i)}$ ，它存储到目前为止的部分和
- 将块  $y_{(i-t) \bmod N}$  发送到设备  $(i+1) \bmod N$
- 从设备  $(i-1) \bmod N$  接收块  $z_{(i-1) \bmod N}$
- 更新部分总和的副本：  $y_{(i-t-1) \bmod N} \leftarrow y_{(i-t-1) \bmod N} + z_{(i-1) \bmod N}$

在  $N-1$  个步骤之后，设备  $i$  就获得了块  $y_i$  的全部总和，因此，ringreduce-scatter 需要  $\frac{N-1}{N} \frac{S}{W}$  秒，就像 ring all-gather 一样。

最后，我们来实现环all-reduce。在全归约中，每个设备  $i$  以大小为  $S$  的完整张量  $x^{(i)}$  开始，并以归约  $y = \sum_{i=0}^{N-1} x^{(i)}$  结束。我们将把all-reduce实现为一个环reduce-scatter，然后是一个环all-gather，所以环all-reduce需要  $2 \frac{N-1}{N} \frac{S}{W}$  秒。

问题 (`alternate_ring_all_reduce`) : 交替环全归约 (1 分)

让我们使用以下算法，而不是将 all-reduce 实现为环reduce-scatter 和环 all-gather:

对于步骤  $t = 1, \dots, N - 1$ ，设备  $i$  执行以下操作:

- 如果是  $t = 1$ ，则初始化  $y \leftarrow x^{(i)}$ ，它存储到目前为止的部分和
- 将  $x^{((i-t+1) \bmod N)}$  发送到设备  $(i + 1) \bmod N$
- 从设备  $(i - 1) \bmod N$  接收  $x^{((i-t) \bmod N)}$
- 更新部分总和的副本:  $y \leftarrow y + x^{((i-t) \bmod N)}$

在与上述相同的设置中（每个设备的  $W$  出口带宽，每个  $x^{(i)}$  的大小为  $S$ ），该算法需要多长时间？

可交付成果：根据  $S$ 、 $N$  和  $W$  给出答案，并附上一句话理由。

## 8.2 并行分析数据

考虑到我们的通信原语，我们准备分析并行策略。我们将分析单个 FFN 层的并行化。回想一下，给定输入  $x$ ，我们的前向传播由以下公式给出：

$$x_1 = xW_1 \quad (20)$$

$$x_2 = xW_2 \quad (21)$$

$$z = f(x_1) * x_2 \quad (22)$$

$$y = zW_3, \quad (23)$$

其中  $x$  具有形状  $(B, D)$ ， $W_1$  和  $W_2$  具有形状  $(D, D_{\text{FF}})$ ，而  $W_3$  具有形状  $(D_{\text{FF}}, D)$ 。  $f$  是我们的元素激活函数（例如 SiLU）， $*$  表示元素乘法。

显式地写出向后传递也很有用。回想一下，给定形状为  $(B, D)$  的  $dy$ ，后向传播由以下公式给出：

$$dz = dyW_3^\top \quad (24)$$

$$dx_2 = dz * f(x_1) \quad (25)$$

$$dx_1 = dz * f'(x_1) * x_2 \quad (26)$$

$$dx = dx_1W_1^\top + dx_2W_2^\top \quad (27)$$

$$dW_3 = z^\top dy \quad (28)$$

$$dW_2 = x^\top dx_2 \quad (29)$$

$$dW_1 = x^\top dx_1, \quad (30)$$

其中  $*$  表示元素乘法。

回想一下，在使用  $N_{\text{DP}}$  设备的数据并行中，我们将输入  $x$  分成大小为  $\left(\frac{B}{N_{\text{DP}}}, D\right)$  的分片  $x^{(i)}$ 。前向传递照常进行，没有任何集合，产生大小为  $\left(\frac{B}{N_{\text{DP}}}, D\right)$  的激活  $y^{(i)}$ 。在

向后传递，像往常一样进行批量分片激活，设备  $i$  最终得到梯度

$$dW_3^{(i)} = z^{(i)\top} dy^{(i)} \quad (31)$$

$$dW_2^{(i)} = x^{(i)\top} dx_2^{(i)} \quad (32)$$

$$dW_1^{(i)} = x^{(i)\top} dx_1^{(i)}, \quad (33)$$

其中，我们不是对所有  $B$  外部产品求和，而是仅对输入分片的  $N_{DP}^B$  外部产品进行部分求和。然后，我们需要跨设备进行 all-reduce 以获得完整的梯度  $dW_3$ 、 $dW_2$  和  $dW_1$ 。

问题 (**data\_parallel\_calcs**)： 数据并行计算 (3分)

当数据并行性成为通信瓶颈时，我们现在拥有计算所需的一切。令  $C$ （以 FLOP/s 为单位）表示设备加速器速度，而  $W$ （以每秒字节数为单位）表示每个设备的出口带宽。然后我们可以计算计算时间和通信时间。由于计算和通信可以重叠，因此当通信时间大于计算时间时，我们就会遇到瓶颈。我们假设所有权重和激活都是 FP16（即两个字节）。

(a) 使用  $N_{DP}$  数据并行性计算反向传递需要多少 FLOP？ 您可以忽略所有非 matmul 运算。回想一下， $\text{matmul}(A, B)(B, C) \rightarrow (A, C)$  需要  $2ABC$  次失败。

可交付成果：以  $B$ 、 $D$ 、 $D_{FF}$  和  $N_{DP}$  形式给出答案，并附上一句话理由。

(b) 在  $N_{DP}$  数据并行度下，向后传递需要多少通信时间？

可交付成果：以  $B$ 、 $D$ 、 $D_{FF}$ 、 $N_{DP}$  和  $W$  的子集形式表示的答案，以及一句话论证。

(c) 固定其他参数，在我们出现通信瓶颈之前， $N_{DP}$  可以变得多大？

可交付成果：一侧为  $N_{DP}$  的不等式，另一侧为  $B$ 、 $D$ 、 $D_{FF}$ 、 $C$  和  $W$  子集的表达式，以及单句论证。

### 8.3 并行分析完全分片数据

接下来我们分析一下FSDP。回想一下，就像 DP 一样，FSDP 对输入和激活的批量维度进行分片。此外，为了节省内存，我们还对优化器状态、梯度和权重进行了分片。我们可以沿任一维度对权重进行分片，在设备  $i$  上生成分片  $W_1^{(i)}$ 、 $W_2^{(i)}$  和  $W_3^{(i)}$ ，每个分片的大小为  $\frac{DD_{FF}}{N_{FSDP}}$ 。

在前向传递中，我们将仅对批量分片输入进行数据并行前向传递。但要做到这一点，请记住，我们首先需要跨设备收集所有权重碎片：

$$W_1 = \text{all-gather} \left( \left\{ W_1^{(i)} \right\}_{i=0}^{N_{FSDP}-1} \right) \quad (34)$$

$$W_2 = \text{all-gather} \left( \left\{ W_2^{(i)} \right\}_{i=0}^{N_{FSDP}-1} \right) \quad (35)$$

$$\mathbf{W}_3 = \text{all-gather} \left( \left\{ \mathbf{W}_3^{(i)} \right\}_{i=0}^{N_{\text{FSDP}}-1} \right) \quad (36)$$

$$(\text{do batch-sharded forward pass}) \quad (37)$$

请注意，当我们想要进行全部收集时，我们有一定的自由度：我们只需要在使用之前获得全部收集的权重，然后我们应该丢弃它以保持较低的内存成本。为简单起见，在本节中我们仅将三个全聚集一起列出。

与前向传递一样，在后向传递中，我们需要首先跨设备收集权重碎片。然后我们可以进行数据并行向后传递，但我们不再需要进行全归约，因为每个设备只需要其分片的梯度。因此，我们进行减少分散：

$$\mathbf{W}_1 = \text{all-gather} \left( \left\{ \mathbf{W}_1^{(i)} \right\}_{i=0}^{N_{\text{FSDP}}-1} \right) \quad (38)$$

$$\mathbf{W}_2 = \text{all-gather} \left( \left\{ \mathbf{W}_2^{(i)} \right\}_{i=0}^{N_{\text{FSDP}}-1} \right) \quad (39)$$

$$\mathbf{W}_3 = \text{all-gather} \left( \left\{ \mathbf{W}_3^{(i)} \right\}_{i=0}^{N_{\text{FSDP}}-1} \right) \quad (40)$$

$$(\text{do batch-sharded backward pass}) \quad (41)$$

$$d\mathbf{W}_1^{(i)} = \text{reduce-scatter} \left( \left\{ d\mathbf{W}_1^{(i)} \right\}_{i=0}^{N_{\text{FSDP}}-1} \right) \quad (42)$$

$$d\mathbf{W}_2^{(i)} = \text{reduce-scatter} \left( \left\{ d\mathbf{W}_2^{(i)} \right\}_{i=0}^{N_{\text{FSDP}}-1} \right) \quad (43)$$

$$d\mathbf{W}_3^{(i)} = \text{reduce-scatter} \left( \left\{ d\mathbf{W}_3^{(i)} \right\}_{i=0}^{N_{\text{FSDP}}-1} \right) \quad (44)$$

请注意，分片符号（上标  $(i)$ ）已过载：reduce-scatter 的输入是完整权重的部分和，而输出是分片权重的完整和。

问题（**fsdp\_calcs**）： 完全分片数据并行计算（3分）

在与数据并行计算相同的设置下，我们来计算一下FSDP何时会出现通信瓶颈。

(a) 使用  $N_{\text{FSDP}}$  FSDP 计算反向传播需要多少 FLOP？那么前传呢？可交付成果：两个关于  $B$ 、 $D$ 、 $D_{\text{FF}}$  和  $N_{\text{FSDP}}$  的答案，以及两个一句话论证。(b) 使用  $N_{\text{FSDP}}$  FSDP，后向传播需要多少通信时间？那么前传呢？可交付成果：根据  $B$ 、 $D$ 、 $D_{\text{FF}}$ 、 $N_{\text{FSDP}}$  和  $W$  的子集提供两个答案，以及两个单句理由。(c) 固定其他参数，在向后传递出现通信瓶颈之前， $N_{\text{FSDP}}$  可以变得多大？那么前传呢？

可交付成果：一侧为  $N_{\text{FSDP}}$  的两个不等式，另一侧为  $B$ 、 $D$ 、 $D_{\text{FF}}$ 、 $C$  和  $W$  子集的表达式，以及两个单句论证。

## 8.4 分析张量并行

在实践中，FSDP 通常与称为张量并行（TP）的并行策略相结合。在 TP 中，我们跨设备分割每个权重矩阵的输入或输出维度。输入维度分片通常称为“行并行”，而输出维度分片通常称为“列并行”。

具体来说，假设我们想要对  $\text{matmul } \mathbf{x}\mathbf{W}$  进行分片，其中  $\mathbf{x}$  的形状为  $(B, D)$ ， $\mathbf{W}$  的形状为  $(D, D_{\text{FF}})$ 。在列并行中，我们有形状为  $(D, \frac{D_{\text{FF}}}{N_{\text{TP}}})$  的分片  $\mathbf{W}^{(i)}$ ，并且我们有 —

$$\mathbf{x}\mathbf{W} = \text{all-gather} \left( \left\{ \mathbf{x}\mathbf{W}^{(i)} \right\}_{i=0}^{N_{\text{TP}}-1} \right). \quad (45)$$

另一方面，对于行并行，我们有形状为  $(\frac{D}{N_{\text{TP}}}, D_{\text{FF}})$  的分片  $\mathbf{W}^{(i)}$ ，并且每个设备在进行  $\text{matmul}$  之前也会将输入  $\mathbf{x}$  缩小为形状为  $(B, \frac{D}{N_{\text{TP}}})$  的切片  $\mathbf{x}^{(i)}$ 。那么，我们有

$$\mathbf{x}\mathbf{W} = \text{all-reduce} \left( \left\{ \mathbf{x}^{(i)}\mathbf{W}^{(i)} \right\}_{i=0}^{N_{\text{TP}}-1} \right). \quad (46)$$

为了并行化我们的 FFN，我们将使用特定的张量并行配置，其中  $\mathbf{W}_1$  和  $\mathbf{W}_2$  是列并行（输出维度分片），而  $\mathbf{W}_3$  是行并行（输入维度分片）。由于行并行权重仅需要输入的一部分，因此此配置让我们可以跳过列并行权重之后的全部收集。给定大小为  $(B, D)$  的输入  $\mathbf{x}$ ，该策略给出以下前向传递：

$$\mathbf{x}_1^{(i)} = \mathbf{x}\mathbf{W}_1^{(i)} \quad (47)$$

$$\mathbf{x}_2^{(i)} = \mathbf{x}_1^{(i)}\mathbf{W}_2^{(i)} \quad (48)$$

$$\mathbf{z}^{(i)} = f(\mathbf{x}_1^{(i)}) * \mathbf{x}_2^{(i)} \quad (49)$$

$$\mathbf{y}^{(i)} = \mathbf{z}^{(i)}\mathbf{W}_3^{(i)} \quad (50)$$

$$\mathbf{y} = \text{all-reduce} \left( \left\{ \mathbf{y}^{(i)} \right\}_{i=0}^{N_{\text{TP}}-1} \right), \quad (51)$$

其中  $\mathbf{W}_1^{(i)}$  和  $\mathbf{W}_2^{(i)}$  具有形状  $(D, \frac{D_{\text{FF}}}{N_{\text{TP}}})$ ，而  $\mathbf{W}_3^{(i)}$  具有形状  $(\frac{D_{\text{FF}}}{N_{\text{TP}}}, D)$ 。

问题（tp\_calcs）： 张量并行计算（4分）

在与 DP 和 FSDP 计算相同的设置下，我们来计算一下 TP 何时变为沟通出现瓶颈。(a) 给定大小为  $(B, D)$  的输入  $\mathbf{dy}$ ，写出上述张量并行策略的向后传递（其中  $\mathbf{W}_1^{(i)}$  和  $\mathbf{W}_2^{(i)}$  具有形状  $(D, \frac{D_{\text{FF}}}{N_{\text{TP}}})$ ， $\mathbf{W}_3^{(i)}$  具有形状  $(\frac{D_{\text{FF}}}{N_{\text{TP}}}, D)$ ）。可交付成果：一系列描述后向传递的方程，包括  $\mathbf{dy}$ 、分片权重  $(\mathbf{W}_1^{(i)}, \mathbf{W}_2^{(i)}, \mathbf{W}_3^{(i)})$ 、前向传递保存的激活值  $(\mathbf{x}, \mathbf{x}_1^{(i)}, \mathbf{x}_2^{(i)}, \mathbf{z}^{(i)}, \mathbf{y}^{(i)})$ 、通信原语以及您想要定义的任何中间变量。这些方程应该产生每个设备的梯度  $d\mathbf{W}_1^{(i)}$ 、 $d\mathbf{W}_2^{(i)}$ 、 $d\mathbf{W}_3^{(i)}$  和后向传递输出  $d\mathbf{x}$ 。请随意引用第 8.2 节中的非分片反向传递并对其进行修改。(b) 使用  $N_{\text{TP}}$  TP 计算前向传播需要多少次 FLOP？那么向后传球呢？

可交付成果：两个关于  $B$ 、 $D$ 、 $D_{\text{FF}}$  和  $N_{\text{TP}}$  的答案，以及两个单句论证。(c) 对于  $N_{\text{TP}}$  TP，前向传递需要多少通信时间？那么向后传球呢？可交付成果：根据  $B$ 、 $D$ 、 $D_{\text{FF}}$ 、 $N_{\text{TP}}$  和  $W$  的子集提供两个答案，以及两个单句理由。(d) 固定其他参数，在向后传递出现通信瓶颈之前， $N_{\text{TP}}$  可以变得多大？那么前传呢？可交付成果：一侧带有  $N_{\text{TP}}$  的两个不等式，以及子集的表达式

另一方面是  $B$ 、 $D$ 、 $D_{\text{FF}}$ 、 $C$  和  $W$ ，以及两个单句论证。

## 8.5 2D 并行性 (FSDP + TP)

我们终于准备好结合并行策略了！在本节中，我们将了解如何最佳地结合 FSDP 和 TP。提示：在前面的部分中，您应该已经发现批量大小和模型大小参数限制了您可以扩展到的设备数量，其中使所有内容变大可以让您继续扩展设备而不会遇到通信瓶颈。不幸的是，将批量大小调整到一定程度后就会开始降低性能，因为梯度噪声显着缩小，从而失去了 SGD 的隐式正则化属性；这一点通常称为“临界批量大小”。缩放定律通常告诉我们我们希望模型有多大（这将是你在下一个任务中的工作！）。

在本节中，我们将考虑一种简化的设置，其中有人会向您提供所有问题参数（批量大小、模型大小、带宽、加速器速度）。您的工作将是选择可扩展到尽可能多的设备的 FSDP 和 TP 配置，同时保持计算限制而不是通信限制。

我们首先来了解一下 FSDP 与 TP 结合的机制。每个设备将具有 TP 等级  $i = 0, \dots, N_{\text{TP}} - 1$  和 FSDP 等级  $j = 0, \dots, N_{\text{FSDP}} - 1$ ，形成总共有  $N = N_{\text{TP}} N_{\text{FSDP}}$  台设备的 2D 网格。在 TP 之后，我们将首先沿输出维度对  $\mathbf{W}_1$  和  $\mathbf{W}_2$  进行分片，并沿输入维度对  $\mathbf{W}_3$  进行分片。因此，我们必须在激活上插入 TP 风格的 all-reduce。接下来，应用 FSDP，我们将对输入的批量维度进行分片，并且我们还将沿着 TP 未分片的维度进一步对每个权重矩阵进行分片。然后，在进行 TP 式前向/后向传递之前，我们必须在权重上插入 FSDP 式全聚集，并在 TP 式后向传递之后在权重梯度上插入减少分散。

结果是每个设备  $(i, j)$  持有形状为  $\left(\frac{D}{N_{\text{FSDP}}}, \frac{D_{\text{FF}}}{N_{\text{TP}}}\right)$  的权重分片  $\mathbf{W}_1^{(i,j)}$  和  $\mathbf{W}_2^{(i,j)}$ ，以及形状为  $\left(\frac{D_{\text{FF}}}{N_{\text{TP}}}, \frac{D}{N_{\text{FSDP}}}\right)$  的权重分片  $\mathbf{W}_3^{(i,j)}$ 。然后，给定大小为  $\left(\frac{B}{N_{\text{FSDP}}}, D\right)$  的批量分片输入  $\mathbf{x}^{(j)}$ ，我们可以将前向传递写出如下：

$$\mathbf{W}_1^{(i)} = \text{all-gather} \left( \left\{ \mathbf{W}_1^{(i,j)} \right\}_{j=0}^{N_{\text{FSDP}}-1} \right) \quad (52)$$

$$\mathbf{W}_2^{(i)} = \text{all-gather} \left( \left\{ \mathbf{W}_2^{(i,j)} \right\}_{j=0}^{N_{\text{FSDP}}-1} \right) \quad (53)$$

$$\mathbf{W}_3^{(i)} = \text{all-gather} \left( \left\{ \mathbf{W}_3^{(i,j)} \right\}_{j=0}^{N_{\text{FSDP}}-1} \right) \quad (54)$$

$$\mathbf{x}_1^{(i,j)} = \mathbf{x}^{(j)} \mathbf{W}_1^{(i)} \quad (55)$$

$$\mathbf{x}_2^{(i,j)} = \mathbf{x}^{(j)} \mathbf{W}_2^{(i)} \quad (56)$$

$$\mathbf{z}^{(i,j)} = f\left(\mathbf{x}_1^{(i,j)}\right) * \mathbf{x}_2^{(i,j)} \quad (57)$$

$$\mathbf{y}^{(i,j)} = \mathbf{z}^{(i,j)} \mathbf{W}_3^{(i)} \quad (58)$$

$$\mathbf{y}^{(j)} = \text{all-reduce} \left( \left\{ \mathbf{y}^{(i,j)} \right\}_{i=0}^{N_{\text{TP}}-1} \right), \quad (59)$$

最终得到大小为  $\left(\frac{B}{N_{\text{FSDP}}}, D\right)$  的批量分片输出  $\mathbf{y}^{(j)}$ 。为子简洁起见，本节中我们将省略向后传递，只关注向前传递。但此时，您应该拥有自己写出所需的所有信息。

#### 问题 (`fsdp_tp_calcs`) : 2D并行计算 (6分)

在与目前计算相同的设置下，让我们计算一下 2D 并行度何时变为遇到瓶颈。(a) 使用  $N_{\text{FSDP}}$  FSDP +  $N_{\text{TP}}$  TP 计算前向传播需要多少次 FLOP? 可交付成果: 以  $B$ 、 $D$ 、 $D_{\text{FF}}$ 、 $N_{\text{FSDP}}$  和  $N_{\text{TP}}$  形式给出答案，并附上一句话论证。(b) 使用  $N_{\text{FSDP}}$  FSDP +  $N_{\text{TP}}$  TP 时，前向传播需要多少通信时间? 假设沿着每个轴的通信可以重叠 (换句话说，沿着 FSDP 轴的集合可以与沿着 TP 轴的集合重叠)。可交付成果: 以  $B$ 、 $D$ 、 $D_{\text{FF}}$ 、 $N_{\text{FSDP}}$ 、 $N_{\text{TP}}$  和  $W$  的子集形式表示的答案，以及一句话论证。提示: 答案应表示为两个数量 (FSDP 和 TP 集体成本) 之间的最大值，因为两者可以重叠。(c) 在  $N_{\text{TP}}$  和  $N_{\text{FSDP}}$  的最佳设置下，在前向传播出现通信瓶颈之前， $N = N_{\text{TP}} N_{\text{FSDP}}$  可以变得多大? 可交付成果: 一侧为  $N$  的不等式，另一侧为  $B$ 、 $D$ 、 $D_{\text{FF}}$ 、 $C$  和  $W$  子集的表达式，以及一些句子和方程作为论证。(d) 现在假设 FSDP 轴和 TP 轴集合不能重叠，因为它们共享相同的网络资源。在  $N_{\text{TP}}$  和  $N_{\text{FSDP}}$  的最佳设置下，在前向传播出现通信瓶颈之前， $N = N_{\text{TP}} N_{\text{FSDP}}$  可以变得多大? 不用担心将  $N_{\text{TP}}$  和  $N_{\text{FSDP}}$  截断为整数。可交付成果: 一侧为  $N$  的不等式，另一侧为  $B$ 、 $D$ 、 $D_{\text{FF}}$ 、 $C$  和  $W$  子集的表达式，以及一些句子和方程作为论证。

## 9 排行榜

作业 2 的排行榜将测试 8B 模型完整训练步骤的速度。我们挑战您使用您能想到的任何技巧来对您的代码进行基准测试并优化内存和运行时。关键限制是您无法更改模型的输入/输出行为。您的实现将针对 `cs336_basics` 目录中的模型进行测试。您的输入将在 BF16 上通过因果屏蔽进行测试，并且它们必须通过与常规实施相同的测试。该实现也必须是您自己的，并且您不能使用或复制预先存在的实现。您的计时应使用批量大小为 2、序列长度为 32,768 的样本在两个 B200 GPU 上进行测量。故意使模型难以适应内存。有关完整配置，请参阅下面的代码。我们

将验证前 5-10 名提交的正确性和性能。我们将运行以下测试来计时您的实施：

```
class Config:
    ctx_len = 32768
    vocab_size = 151936
    d_model = 4096
    d_ff = 11008
    num_layers = 34
    num_heads = 32
    torch_dtype = torch.bfloat16
    is_causal = True
    batch_size = 2
cfg = Config()

def test_timing_forward_backward():
    labels, targets = torch.randint(high=cfg.vocab_size, size=(2, cfg.batch_size,
    cfg.ctx_len))

    model = BasicsTransformerLM(Config())
    optimizer = AdamW(model.parameters())

    def train_step():
        optimizer.zero_grad(set_to_none=True)
        res = model(labels)
        loss = cross_entropy(res, targets).sum()
        loss.backward()
        optimizer.step()

    timing_results = triton.testing.do_bench(train_step, rep=30_000, warmup=10_000)
    print(timing_results)
```

出于测试目的，您可以将重复时间和预热时间（以毫秒为单位）缩短到更短的时间。

一些改进和确保模型适合的想法：

- 调整内核的图块大小（为此使用 Triton 自动调整！）
- 调整额外的 Triton/torch.compile 配置参数
- 实施融合 AdamW
- 基本实现正在具体化完整的 logits ([batch, seq\_len, vocab\_size])。编写一个融合 LM 头和交叉熵损失的内核。您还可以让它以融合的方式立即计算向后传递
- 改进 FlashAttention 在 Triton 中实现反向传递，而不仅仅是 torch.compile（参见第 4.2.3 节）对反向传递的输入进行两次传递，一次用于 dQ，另一次用于 dK 和 dV，以避免块之间的原子或同步 在进行因果屏蔽时尽早停止程序实例，跳过保证全为零的图块 将非屏蔽图块与图块分开对角线，在不比较索引的情况下计算第一个，通过一次比较计算第二个

在 Hopper 之后的架构上使用 TMA（张量内存加速器）功能，遵循与我们的教程类似的模式

- 仅在需要时才使用激活检查点来以运行时速度换取内存节省

问题 (Leaderboard): 排行榜: 最快的训练步骤 (10 分)

该基准测试将以批量大小 2 在两个 B200 GPU 上运行。您提交的内容将根据完整的训练步骤的挂钟时间进行评估: 前向传递、损失、后向传递和 AdamW 更新。

从空的 PyTorch/Triton 缓存中, 您的基准测试运行必须在 10 分钟内完成, 因此请小心过度激进的 `torch.compile` 和 Triton 自动调整。

可交付成果: 使用 AdamW 进行完整的前向和后向训练步骤的最佳挂钟时间。

我们预计排行榜提交时间将超过 10 秒的初始基线。

将您的结果提交到此处的排行榜:

[github.com/stanford-cs336/assignment2-systems-leaderboard](https://github.com/stanford-cs336/assignment2-systems-leaderboard)

## 参考书目

[1] T. Dao, “FlashAttention-2: 更快的注意力, 更好的并行性和工作分区。” [在线的]。

可用: <https://arxiv.org/abs/2307.08691> [2] T. Dao、D. Y. Fu、S. Ermon、A. Rudra 和 C. Re, “FlashAttention: 快速且内存高效的精确

Attention with IO-Awareness,” in *Advances in Neural Information Processing Systems*, A. H. Oh、A. Agarwal、D. Belgrave 和 K. Cho, Eds., 2022。[在线]。可用: <https://openreview.net/forum?id=H4DqfPSibmx> [3] M. Milakov 和 N. Gimelshein, “softmax 的在线标准化器计算”。[在线]。可用: <https://arxiv.org/abs/1805.02867> [4] H. He, “让深度学习从第一原理走向辉煌”, 2022 年, [在线]。可用: [https://horace.io/brr\\_intro.html](https://horace.io/brr_intro.html) [5] S. Rajbhandari、J. Rasley、O. Ruwase 和 Y. He, “ZeRO: 训练万亿参数模型的内存优化。” 2020 年。[6] J. Austin *et al.*, “如何扩展模型”, 2025 年。[7] H. Z. P. N. M. M. L. W. T. W. Nouamane Tazi Ferdinand Mom, “超大规模手册: 在 GPU 集群上训练法学硕士。” 2025 年。